



**Uncertainty Reasoner applied to Real-World
Numismatic RDF Graphs**

Bachelor Thesis

Presented by

Oussama Izelgue

Submitted to

Dr. Karsten Tolle

For the degree

Bachelor of Science Informatik

Fachbereich 12 Mathematik und Informatik

Johann Wolfgang Goethe-Universität Frankfurt am Main

Abstract

The identification of ancient coin issuers and depicted persons is often complicated by significant uncertainty, due to factors such as corrosion, incomplete provenance, and conflicting expert opinions [12]. In this context, RDF graphs are used to represent and connect complex data from multiple sources in a structured, machine-readable form [6]. However, effectively modeling and reasoning about uncertainty within RDF graphs remains a challenge, making it necessary to develop robust approaches that can handle ambiguous and incomplete information for improved analysis and decision-making.

This thesis evaluates *Uncertainty Reasoner*—a reasoner that was first developed and presented in earlier research [13]—by applying it to real-world numismatic data from the AFE relational database. The evaluation begins with an analysis of interaction patterns between coin issuer and depicted-person inputs from the AFE web form, which leads to the identification of three necessary improvements to *Uncertainty Reasoner*. The thesis demonstrates how these improvements enhance the quality of results generated in critical situations, and increase the reasoner’s flexibility and the reliability of reasoning outcomes for real-world numismatic RDF graphs.

Furthermore, in order to maintain the fundamental design principles established in earlier research [13], the suggested enhancements were carefully designed to maintain the reasoner’s original specifications, which included simplicity, expandability, and ease of use.

Contents

1	Introduction	1
2	Background and Theoretical Foundations	3
2.1	Semantic Web	3
2.2	RDF Fundamentals	3
2.3	RDFS and OWL in Brief	6
2.4	Uncertainty Modelling in RDF graphs	7
2.5	Dempster-Shafer theory: The Dempster rule of combination	8
3	Data and Methods: AFE and Uncertainty Reasoner	11
3.1	AFE Data and Uncertainty Capture: Issuer & Depicted Person	11
3.2	Uncertainty Reasoner: Architecture and Axioms	14
3.2.1	Data Representation	14
3.2.2	Uncertainty Reasoner's Axioms	17
3.2.2.1	CertaintyAssignmentAxiom	17
3.2.2.2	AFEDempsterShaferAxiom	19
4	Classification of AFE Web-Form Input Patterns: issuer & issuing-for	22
4.1	Issuer & Depicted Person as One Integrated Use Case: 16 input patterns	22
4.2	Further analysis: Positive and negative use cases	24
5	Enhancement of Uncertainty Reasoner	26
5.1	First Enhancement: Prioritizing Expert Knowledge in special Negative Cases	26
5.1.1	Context and Issue	26
5.1.2	Proposed Approach	28

5.1.2.1	Assumptions and setting	30
5.1.2.2	Rule for Domain-Knowledge Ignorance in negative cases	30
5.1.2.3	Proof of the Rule	30
5.1.2.4	Evaluation	31
5.1.3	Implementation Details	32
5.2	Second Enhancement: Applying AFEDempsterShaferAxiom to the coin's depicted person	35
5.2.1	Context and Issue	35
5.2.2	Proposed Approach	37
5.2.3	Implementation Details	39
5.3	Third Enhancement: Parallel Execution of the AFEDempsterShaferAxiom	41
5.3.1	Context and Issue	41
5.3.1.1	Experiment 1	42
5.3.1.2	Experiment 2	44
5.3.1.3	Problem Summary	46
5.3.2	Proposed Approach	47
5.3.3	Implementation Details	48
5.3.4	Limitations	50
6	Conclusion	51
	Appendices	57
A.	RDF Namespaces	57

1 Introduction

The Semantic Web aims to represent knowledge in a structured and abstract way, so that computers can reason over it, infer meaningful conclusions and exchange information, ultimately enabling the distribution and reconciliation of knowledge on a global scale [6]. As a foundational technology within the Semantic Web, the Resource Description Framework (RDF) is a formal language designed for describing structured information, enabling the exchange and integration of data on the web without loss of its original meaning [7].

Ancient numismatics presents a unique set of challenges for digital representation and analysis. The identification of coins is frequently complicated by uncertainties arising from different factors such as corrosion, wear coins, even diverging interpretations among experts [12]. As a result, numismatic collections are often characterized by uncertain and incomplete data, complicating precise attribution and classification [12].

As a result, the Antike Fundmünzen in Europa database (AFE) [1], which records the discovery of ancient coins in Germany, was established [12]. There, numismatists can use the AFE web-form to enter information on finds of coins and their attributes, some of which are measurable such as the coin’s size and others that are not such as the coin’s depicted person or issuer. Most crucial though, is that the user can always enter some alternatives to these attributes and indicate whether he is uncertain of his choices [12].

Recent research [13] has explored various theoretical methodologies to handle and reason over uncertainty in RDF graphs. The outcome of the research was *Uncertainty Reasoner*, a reasoner developed to allow its users to define personalized reasoning approaches and uncertainty modeling techniques within RDF-based knowledge representations [13].

Although *Uncertainty Reasoner* was validated through numismatic use cases, demonstrating correct theoretical implementation, this thesis specifically applies it to real-world numismatic datasets sourced from the AFE database.

The goal is to critically evaluate this reasoner, particularly focusing on potential enhancements to AFEDempsterShaferAxiom—an important component of *Uncertainty Reasoner*—and expand user options for uncertainty management.

The thesis begins with an overview of the relevant background and related work, including Semantic Web technologies, RDF fundamentals, the basic principles of modeling uncertainty and the Dempster-Shafer theory. Subsequent chapters describe how the AFE web form is used to record uncertainty about coin’s issuers and depicted persons, and present the resulting RDF graphs along with *Uncertainty Reasoner*’s architecture and axioms, which form the foundation of its methodology. In the main part of this thesis, an analysis of the interaction patterns between issuer and depicted-person inputs was conducted, resulting in their classification into 16 use cases, which were

further divided into positive and negative cases. Based on this analysis, the thesis then presents three improvements to *Uncertainty Reasoner*, each of which is explained in detail along with the background of the underlying issue, the solution strategy to solve it, and the corresponding code implementation.

The complete implementation developed in the context of this thesis can be found on GitHub¹.

¹<https://github.com/Oussama-Izg/uncertainty-reasoner> (last accessed 25.09.2025).

2 Background and Theoretical Foundations

In this section, we present the theoretical foundation required to comprehend the subsequent sections of this thesis. Subsections 2.1, 2.2 and 2.3, which are based on [6], cover the fundamental ideas of the Semantic Web, the Resource Description Framework (RDF) and its extensions (RDFS, OWL). Subsection 2.4 describes the different methods for modelling uncertainty in RDF graphs and the last subsection 2.5 introduces the Dempster–Shafer Theory, which represents the core of `AFEDempsterShaferAxiom`.

2.1 Semantic Web

The development of Semantic Web technologies has been linked to the evolution of the World Wide Web since Tim Berners-Lee, the inventor of the World Wide Web, first suggested the idea and encouraged research in this area [6].

There are three conceptual foundations or so say concepts that underpin the development of the Semantic Web: first, the development of abstract models that help in the representation and understanding of complex realities; second, the design of intelligent systems that can derive significant conclusions from structured knowledge; and third, facilitation of seamless information exchange between computers [6].

The four principal technologies that help to realize these foundational concepts are Resource Description Framework (RDF), RDFS, OWL, and SPARQL.

2.2 RDF Fundamentals

The Resource Description Framework (RDF) is a formal language designed to describe structured information and its main goal is to facilitate the exchange of data across the web while maintaining the original meaning. While known Markup Languages such as HTML or XML intend mainly to display documents correctly, RDF further focus on the processing and recombination of the information, which is why RDF is often considered the basic representation format for developing the Semantic Web.

An RDF document represents a directed graph, consisting of nodes linked by directed edges where both nodes and edges are labeled with identifiers for differentiation (see Figure 1).



Figure 1: Example RDF graph with URI nodes in the AFE domain.

From this example it might seem logical to use XML which, in contrast to graphs, encodes information in a tree structure, which is often considered ideal for organizing hierarchical information in documents. However, RDF relies on graphs for specific reasons. Firstly, RDF was not intended for document structuring but rather for describing general relationships between objects, referred to as *resources* in RDF. For instance, a graph might express that a coin labeled as `coin_7` has a portrait of a person called `Domitianus`, where the relationship between the coin and the depicted person does not fit hierarchically under either resource. RDF treats such relationships, on the other hand, as fundamental building blocks of information, naturally forming graphs rather than hierarchical trees. Second, integrating decentralized web data from various sources is made easier with RDF graphs. Graphs are easy to combine because, even if some of their components are disconnected, they all combine to form a larger graph. On the other hand, XML document merging is difficult because it may scatter related data and not always produce a valid tree. RDF graphs are therefore better suited for integrating distributed information.

When integrating distributed information—RDF graphs—a key challenge is the reliable identification of resources. For instance, suppose one graph from a particular source contains information about a coin labeled as `coin_7`, while another graph from a different source has additional details about the same coin. Therefore, the question here is how can we be sure that both graphs indeed refer to the exact same coin. RDF addresses this issue by using Uniform Resource Identifiers (URIs), which are globally unique, and therefore give each resource a clear and unique identifier to prevent misunderstandings. URIs are used to name not only entities such as coins and people, but also the relationships between them. For example, a specific coin like `coin_7` might have the URI `http://example.org/coin_7`, a person depicted on a coin such as `Domitianus` might be identified by `http://example.org/Domitianus`, and the relationship between a coin and its portrait could be expressed as `http://nomisma.org/ontology#hasPortrait`.

As we can see in Figure 1, even when describing something simple, using full URIs can take up a lot of space. So, as graphs become more complex and the number or length of URIs increases, it quickly becomes difficult for humans to read and understand the graph. This is where namespaces play an important role: URIs are shortened with prefixes in the form “prefix:”. For example, the namespace `http://www.w3.org/1999/02/22-rdf-syntax-ns#` is abbreviated with the prefix `rdf`, and in our case, `http://nomisma.org/ontology#` is abbreviated as `nmo`. So, by using these prefixes, the graphs become much more readable and manageable (see Figure 2).

Throughout the rest of this thesis, only prefixes will be used in graphs and figures instead of writing out full URIs. A list of all namespaces and prefixes used can be found in Table 26 in the appendix.

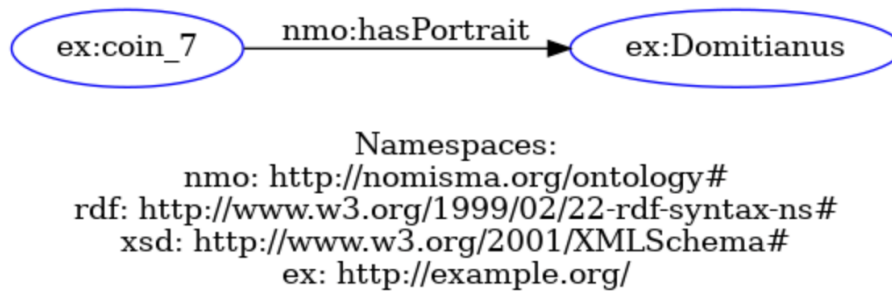


Figure 2: Same RDF graph as in Figure 1, expressed with prefixes instead of full URIs

To express specific data values, RDF graphs can also contain literals, such as strings or numbers. The weight of a coin, for example, might be written as the literal “7.35”, which will always be interpreted to be the same number. This guarantees that specific measurements and facts are interpreted uniformly across different applications.

Sometimes, RDF uses blank nodes which are anonymous and do not have a global identifier (URI). These types of nodes are especially useful for representing intermediate details or data structures that do not need unique names across datasets. This helps avoid creating unnecessary URIs for temporary or unknown resources. These blank nodes are very useful for us when it comes to representing uncertainty in RDF graphs (see subsection 2.4).

Diagrams by themselves are insufficient for processing and storing RDF data. Instead, we use text-based formats, representing the graphs as a set of triples where each triple consists of a subject, predicate, and an object. The most commonly used formats for writing these triples are **Turtle** and **RDF/XML**.

Turtle is a compact and easy-to-read format that uses prefixes to shorten long URIs. For example, in Figure 3, we can see the same graph from Figure 2 represented as Turtle triples.

```

@prefix nmo: <http://nomisma.org/ontology#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix ex: <http://example.org/> .

ex:coin_7 nmo:hasPortrait ex:Domitianus .

```

Figure 3: Turtle serialization of the RDF graph from Figure 2

However, one limitation of Turtle format is that not all programming languages provide built-in support for it. In contrast, RDF/XML is considered the primary syntax for RDF because XML is widely supported across different programming environments. The RDF/XML format does have certain drawbacks, though, such as being less readable overall than Turtle and often requiring careful handling of namespaces and special

characters (see Figure 4).

```
<?xml version="1.0"?>
<rdf:RDF
  xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:nmo="http://nomisma.org/ontology#"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema#"
  xmlns:ex="http://example.org/">

  <rdf:Description rdf:about="http://example.org/coin_7">
    <nmo:hasPortrait rdf:resource="http://example.org/Domitianus"/>
  </rdf:Description>

</rdf:RDF>
```

Figure 4: RDF/XML serialization of the RDF graph from Figure 2

2.3 RDFS and OWL in Brief

As the previous section shows, RDF serves as the basis for describing relationships and structured data, but there are other languages that allow for more expressive knowledge modelling and richer semantics.

RDF Schema (RDFS) builds upon RDF by adding fundamental ontological elements like classes, subclasses, and properties. For example, `ex:RomanCoin` can be defined as a subclass of `ex:Coin` using `rdfs:subClassOf`. This enables the creation of simple vocabularies and lightweight ontologies. Yet, RDFS’s expressive capability remains limited, which makes it ideal for simple applications but not sufficient for expressing deeper domain constraints.

For such advanced requirements, the Web Ontology Language (OWL) has been a W3C-recommended standard for ontology modeling since 2004. OWL adds features like class equivalence, disjointness, property characteristics, and cardinality restrictions. As an example, `owl:cardinality` could be used to specify that the property `nmo:hasPortrait` has a cardinality of 1 for `ex:Coin`, indicating that typically only one person can be represented on a coin. OWL offers three sublanguages—OWL Lite, OWL DL, and OWL Full—offering a choice between different degrees of expressivity, with OWL Lite as the most basic and limiting, OWL DL allowing for more expressivity while maintaining computational completeness, and OWL Full providing the highest level of expressivity but sacrificing decidability.

Although RDFS and OWL are useful for modeling more complex domain knowledge, this thesis focuses on simple RDF graphs and does not discuss RDFS or OWL in further detail, as they are not required to address the specific objectives of this thesis.

2.4 Uncertainty Modelling in RDF graphs

Recent research [8] has systematically compared various approaches for modeling uncertainty in RDF graphs and provided practical benchmarks to identify which methods are most effective under different conditions.

Based on these benchmark tests, the author in [13] designed *Uncertainty Reasoner* to support the reification approach, and particularly the RDF-star approach. In Figure 5, both strategies are demonstrated.

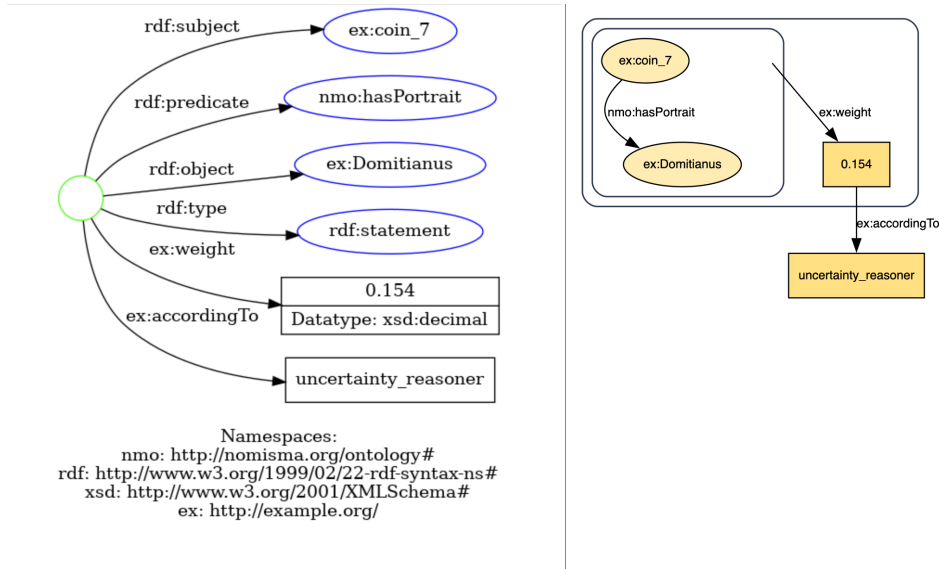


Figure 5: Uncertainty modeling in RDF using Reification (left) and RDF-star (right).

In the reification approach [9]—the standard approach—for a given triple, such as « `ex:coin_7 nmo:hasPortrait ex:Domitianus` » in the example from Figure 5 on the left side, we assign a certainty value by reifying the triple; that is, by representing it as a separate resource using a blank node. This blank node is explicitly typed as an RDF statement with `rdf:type rdf:Statement`, and described by the properties `rdf:subject`, `rdf:predicate`, and `rdf:object`, which correspond to the elements of the original triple. The certainty value is then attached to the blank node as an additional property— `ex:weight`. Additionally, *Uncertainty Reasoner* adds another property named `ex:accordingTo` that indicates the name of the model that assigned that certainty value.

On the other hand, RDF-star, which is an extension of RDF, provides an easier way to make statements about statements [10]. Instead of reifying the triple, RDF-star enables directly embedding the original triple as the subject of other triples. This makes it possible to assign the certainty value directly to the embedded triple, and then to further describe this certainty statement—for example, by indicating the model that

provided the certainty value. As shown in Figure 5 on the right side, the certainty value "0.154" with the property `ex:weight` is attached to the embedded RDF-star triple « `ex:coin_7 nmo:hasPortrait ex:Domitianus` », and the model used to assign this certainty value is then linked to the certainty statement itself using another embedded RDF-star triple, which results in the final representation as follows:

```
« « ex:coin_7 nmo:hasPortrait ex:Domitianus » ex:weight "0.154" »
ex:accordingTo ex:uncertainty_reasoner.
```

Therefore, in contrast to the standard reification approach, which requires at least five triples to represent uncertainty, RDF-star allows for a structure that is simpler, easier to read, and more effective.

According to benchmark results from [8], RDF-star outperforms the reification approach in terms of speed and memory usage, in addition to requiring fewer triples. However, it is important to note that RDF-star is not currently an official W3C standard but rather a Community Group specification maintained by the RDF-DEV Community Group [10].

2.5 Dempster-Shafer theory: The Dempster rule of combination

According to the authors in their research [11], the Dempster-Shafer theory is a mathematical theory of evidence that offers an alternative to standard probability theory for modeling uncertainty. Its main contribution is that it gives probability masses to sets or ranges of possibilities and not just to individual outcomes. Compared to traditional probability, this approach does not require specifying the exact probabilities of each individual element within those sets.

In their paper they also presented the three primary functions that underpin this theory, and they defined them as follows:

- **The Basic Probability Assignment bpa (m):**

The basic probability assignment does not refer to probability in the classical sense. It defines a mapping from the power set $\mathcal{P}(X)$ to the interval $[0, 1]$ and It satisfies the following conditions:

$$m : \mathcal{P}(X) \rightarrow [0, 1], \quad (1)$$

$$m(\emptyset) = 0, \quad (2)$$

$$\sum_{A \in \mathcal{P}(X)} m(A) = 1. \quad (3)$$

The value $m(A)$ expresses the proportion of evidence supporting that an element of the universal set X belongs to the set A but not to any specific subset of A .

- **The Belief function (Bel):**

The belief in a set A is defined as the sum of all basic probability assignments of subsets B that are contained in A :

$$Bel(A) = \sum_{B | B \subseteq A} m(B). \quad (4)$$

And it represents **the lower bound of the probability interval** associated with A .

- **The Plausibility function (Pl):**

The plausibility of a set A is defined as the sum of all basic probability assignments of sets B that intersect A :

$$Pl(A) = \sum_{B | B \cap A \neq \emptyset} m(B). \quad (5)$$

It represents **the upper bound of the probability interval** associated with A .

Therefore, using these three functions, the exact probability for each set $A \subseteq X$ falls within the following range:

$$Bel(A) \leq P(A) \leq Pl(A).$$

In our case, it is possible to determine an approximate value of the probability for each set A within the previously defined interval. As explained by the author in his research [13], while developing `AFEDempsterShaferAxiom` (see Section 3.2.2.2), only two types of sets are considered: single-element sets and the set of all possible elements, which we refer to as *ignorance*. For single-element sets A , since there is no non-empty subsets that could contribute, their belief value matches their mass assignment $m(A)$. But the plausibility of such sets, is given by the sum of $m(A)$ and the mass $m(\text{ignorance})$ assigned to ignorance. As a result, the true probability of A lies between these two measures, making $m(A)$ a good lower-bound estimate of certainty as illustrated in Equation 6, which is adapted from [13].

$$P(A) \approx Bel(A) = m(A) \quad (6)$$

In the other hand, since *ignorance* contains all possible single-elements of the handled context, its belief is always equal to 1. But the probability of ignorance can be computed as one minus the sum of the probabilities of all one-element subsets. This leads to the following relation, which is also adapted from [13]:

$$P(\text{ignorance}) = 1 - \sum_{A | A \neq \text{ignorance}} P(A) \approx 1 - \sum_{A | A \neq \text{ignorance}} m(A) = m(\text{ignorance}) \quad (7)$$

Therefore, the probability of the ignorance subset indicates, the highest possible deviation between the true probabilities and their lower-bound estimates, which also clarifies why the *ignorance* subset is required.

Another important concept in the Dempster-Shafer theory is the **dempster rule of combination**, which offers a method to combine data from several independent sources [11]. Fundamentally, Dempster's rule can be viewed as a generalization of Bayes' rule, where the agreement between different bodies of evidence is emphasized while conflicting evidence is normalized [11].

In order to formalize this intuition, the rule specifies how two basic probability assignments m_1 and m_2 are combined to create a single joint mass (m_{12}) as follows:

$$m_{12}(A) = \frac{\sum_{B \cap C = A} m_1(B)m_2(C)}{1 - K} \quad \text{when } A \neq \emptyset \quad (8)$$

where

$$K = \sum_{B \cap C = \emptyset} m_1(B)m_2(C) \quad (9)$$

represents the degree of conflict between the sources.

The Dempster rule of combination is therefore an interesting concept when it comes to handling uncertainty using evidence since it generates a new probability assignment that encompasses both the conflict between sources and the supporting evidence for a given set.

3 Data and Methods: AFE and Uncertainty Reasoner

In this section, we provide an overview of the AFE dataset and explain *Uncertainty Reasoner* used in our experiments. Section 3.1 explains the AFE web form and the capture of issuer and portrait data, including uncertainty. Section 3.2 presents *Uncertainty Reasoner*—its data access layer, the RDF modeling of records, its internal representation, and its two preprocessing axioms used in this thesis.

3.1 AFE Data and Uncertainty Capture: Issuer & Depicted Person

As stated in the introduction, we apply *Uncertainty Reasoner* to AFE data and modify it on the basis of those data. In [12], the authors describe their approach to storing uncertain data using a web form through which users, such as numismatists, can enter information about discovered coins. This data is then added to their database, the Antike Fundmünzen in Europa database (AFE).

In this form, users enter specific attributes about a specific ancient coin. These attributes are divided into measured attributes, such as size and weight, and non-measured attributes, such as the person depicted on the coin. For each attribute, the user can indicate low confidence in the selected value by checking the "uncertain" box, which means that the value may not be correct.

This thesis focuses exclusively on two attributes: the **depicted person** and the **issuer**. In the entry form, the “Issuing for” field represents the depicted person on the coin and also allows users to add an additional depicted person if necessary. The “Issuer” field indicates the issuer of the coin, with the option to add up to two alternative issuers. This approach enables users to enter multiple values for the same attribute if they are unable to decide on a single entry.

One important aspect to note in the “Issuing-for” field, is that when the user selects a depicted person in the “issuing-for” field, the form automatically displays, under “Possible Issuers” a set of issuer options that are linked to the chosen depicted person. This set of possibilities reflects domain-specific knowledge created and validated by domain experts, based on the current state of their research. However, the user always has the option either to select one of the suggested issuers in the “Issuer” field or to choose a different one that is not among the suggested possibilities.

Figure 6 illustrates this behavior using a real AFE record, in this case for coin 12365, while Figure 7 showcases the coin image that corresponds to it.

Issuing for	Tiberius (Tiberius) ✕ Possible Issuers: • Augustus (Augustus) • Tiberius (Tiberius)	☐ uncertain
Issuing for alternative	✕	
Issuer	Augustus (Augustus) ✕	☒ uncertain

Figure 6: AFE web-form selections for the coin 12365.



Figure 7: Image of the coin 12365

The user in this real use case selected **Tiberius** as the depicted person and **Augustus** as the issuer and marks the issuer as **uncertain**. Based on curated domain knowledge tied to the chosen depicted person, the AFE web-form proposed **Augustus** and **Tiberius** as possible issuers, from which the user selected **Augustus**.

Consequently, there are five scenarios for both attributes—the issuer and the person depicted:

- **Selection of one without checking the “uncertain” box:**

The user is certain that this is the actual issuer or depicted person.

- **Selection of one and checking the “uncertain” box:**

The user is uncertain that this is the actual issuer or depicted person; it could be someone else.

- **Selection of alternatives without checking the “uncertain” box:**

The user is certain that the true issuer or depicted person is among the selected alternatives.

- **Selection of alternatives and checking the “uncertain” box:**

The user is uncertain that the true issuer or depicted person is among the selected alternatives; it could be none of the selected options but rather someone else.

- **Nothing is selected:**

The issuer or depicted person cannot be identified by the user. This possibility is ignored in this thesis, as it does not contribute to the enhancements made to *Uncertainty Reasoner*.

3.2 Uncertainty Reasoner: Architecture and Axioms

To perform inferences on the graph data within our use cases, this thesis utilizes *Uncertainty Reasoner*, a flexible, multi-stage reasoner originally introduced in [13].

This reasoner provides a generalized, axiom-based framework for knowledge inference on RDF graphs. Axioms are simply the rules used to deduce new triples or update the current triples' certainty values.

The reasoning process of *Uncertainty Reasoner* is divided into three distinct phases: the Pre-Processing stage, the Rule-Based Reasoning stage, and the Post-Processing stage. Additionally, the design is very flexible, allowing for the definition of other axioms for inference and uncertainty tailoring, each of which is assigned to a particular processing stage.

Although the reasoner is capable of complex, iterative rule application, the use cases in this thesis require only a single stage - the Pre-Processing Stage. In this stage axioms are applied once to prepare and refine the data before any iterative logic is applied in the Rule-Based Reasoning stage. Among the pre-processing axioms, in this thesis we use only two: `CertaintyAssignmentAxiom` and `AFEDempsterShaferAxiom`.

In subsection 3.2.1 we discuss the RDF modeling of AFE records along with their internal representation, and in subsection 3.2.2 we explain the two axioms.

3.2.1 Data Representation

Since the data are represented as RDF graphs, the author in [13] included an additional component—`SparqlBaseConnector`—to enable communication with an SPARQL endpoint.

SPARQL is a standardized query language for retrieving and manipulating RDF-based data, with a syntax similar to SQL but operating on fundamentally different graph-based structures [6]. It is supported by RDF databases which are called triple stores such as Apache Jena Fuseki, GraphDB, and AllegroGraph. In this thesis, the triple store Apache Jena Fuseki was used during the implementation and modification of the reasoner.

As mentioned in section 2.4, *Uncertainty Reasoner* supports two approaches for representing RDF graphs with uncertainty: **the reification approach** and **the RDF-star approach**. But because the reasoner was implemented in Python and no Python package at the moment exists that can store triples for both approaches, but only for reification, the author developed an internal representation of the RDF graphs. Therefore, once the reasoner reads the triples from the triple store, they are automatically translated into this internal representation and after the reasoning process is completed, the data is translated back into new triples—corresponding to the chosen approach for

representing uncertainty in RDF graphs—and stored in the triple store.

The **internal graph representation** used by the reasoner is a **Pandas data frame**. Tables 1 and 2 adapted from [13] illustrate how *Uncertainty Reasoner* generally represents RDF graphs as Pandas data frames.

In this thesis, however, the data frame format shown in Table 1 is the only one used. With **s** as the subject, **p** as the predicate, and **o** as the object, each row in this representation (**s**, **p**, **o**, **weight**, **model**) correlates to a triple in the RDF graph. The column **model** optionally stores the model or expert that provided the certainty value, and the column **weight** represents the statement’s certainty value with a default value of 1.

The second representation from Table 2 is not required for the use cases addressed in this thesis, given that class relationships are not included in the RDF graphs used here.

Column	Data Type	Description
s	string	Subject of the triple
p	string	Predicate of the triple
o	string	Object of the triple
weight	float64	Certainty or precision degree from 0 to 1. Triples without a specified weight get the default value 1.
model	string	Name of the model or expert this weight value stems from (optional)

Table 1: Internal schema for representing RDF graphs with uncertainty (adapted from [13]).

Column	Data Type	Description
node	string	The node that is referenced
class	string	The class of the node

Table 2: Internal representation of class relationships in RDF graphs (adapted from [13]).

To make this concrete, Figure 8 translates the web-form inputs from Figure 6 into the corresponding RDF graph. The coin node **afe:12365** is linked to **nm:Tiberius** via **nmo:hasPortrait** and to **nm:Augustus** via **nmo:hasIssuer**. Because the user checked *uncertain* for the issuer, the object **ex:uncertain** is asserted for the subject **afe:12365** with the predicate **nmo:hasIssuer**. It is important to note that the node **ex:uncertain**

does not represent a person; rather, it serves as a marker indicating that the user selected the uncertain option, signifying low confidence in the entered values and the possibility that they may be incorrect.

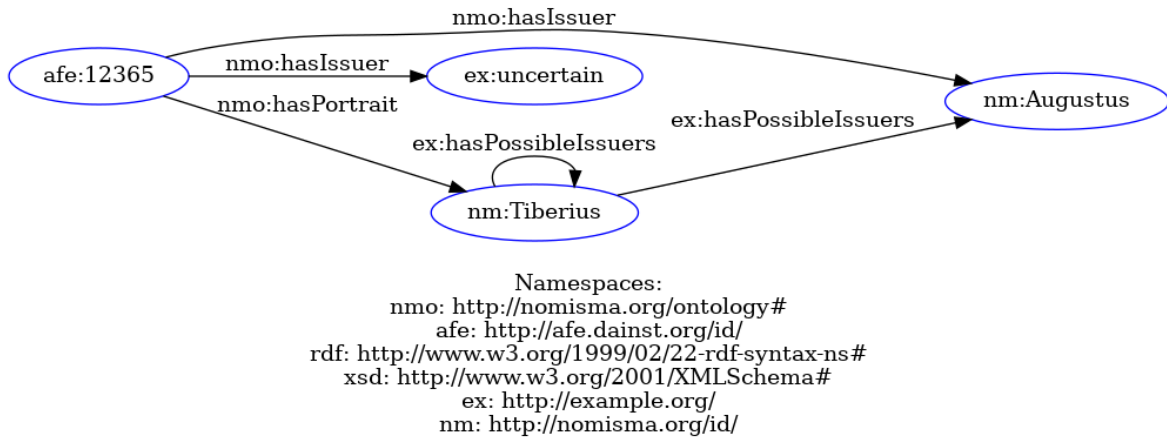


Figure 8: RDF graph representing the entries selected for the AFE record— coin 12365.

The curated domain knowledge presented by the AFE web form is also modeled as edges from the depicted person to the proposed issuers. For example, « nm:Tiberius ex:hasPossibleIssuers nm:Augustus » and « nm:Tiberius ex:hasPossibleIssuers nm:Tiberius » encode these relationships. These edges do not add further facts about the specific coin; rather, they represent expert knowledge about plausible issuers for the chosen depicted person.

When *Uncertainty Reasoner* uploads the Turtle triples from the triple store, each RDF edge is converted into one row of its internal Pandas data frame with the columns **s**, **p**, **o** and the metadata fields **weight** and **model**, as shown before in Table 1.

s	p	o	weight	model
afe:12365	nmo:hasIssuer	nm:Augustus	1	
afe:12365	nmo:hasIssuer	ex:uncertain	1	
afe:12365	nmo:hasPortrait	nm:Tiberius	1	
nm:Tiberius	ex:hasPossibleIssuers	nm:Augustus	1	
nm:Tiberius	ex:hasPossibleIssuers	nm:Tiberius	1	

Table 3: Internal Representaion of the RDF graph from Figure 8 for the coin 12365.

For example, in Table 3, the triples « afe:12365, nmo:hasIssuer, nm:Augustus », « afe:12365, nmo:hasIssuer, ex:uncertain », and « nm:Tiberius, ex:hasPossibleIssuers, nm:Augustus » appear as separate rows. In the data import process, all rows are then initialized with a **weight** of 1 if no certainty values were assigned to them, in order to denote raw pre-reasoning certainty, and the **model** is then left empty, because no reasoning process has yet started. But during reasoning,

these weights could be updated, and the `model` field is then also annotated with the name of *Uncertainty Reasoner* provided by the user of the reasoner.

3.2.2 Uncertainty Reasoner's Axioms

After explaining how user entries for a coin in the AFE web form are converted into RDF graphs, and how the reasoner converts this data stored in a triple store into its internal representation as a Pandas data frame, in this section we present two pre-processing axioms applied by *Uncertainty Reasoner*, originally introduced in [13]. Subsection 3.2.2.1 explains *CertaintyAssignmentAxiom* and subsection 3.2.2.2 explains *AFEDempsterShaferAxiom*.

3.2.2.1 CertaintyAssignmentAxiom

The Python class *CertaintyAssignmentAxiom* uses a straightforward method to assign weights to the given triples.

For a given predicate p , an uncertainty object u (default: `ex:uncertain`), and an uncertainty mass μ (default: 0.2), the triples are first filtered by p . For each subject s (and *model* if it does exist), let n_s be the number of distinct objects with $o \neq u$. If the subject s is not linked to the uncertainty object u , each triple (s, p, o) is assigned a weight of $1/n_s$. If, however, the triple (s, p, u) is present, it is assigned the weight μ , while each of the remaining triples receives $(1 - \mu)/n_s$.

This approach interprets the presence of the uncertainty object as an indication that the provided candidates are possibly not the true ones, uniformly reducing their weights while preserving the total mass of 1 per (s, p) group:

$$\sum_o w(s, p, o) = 1 \quad \text{for each fixed } (s, p).$$

Table 4 illustrates *CertaintyAssignmentAxiom* applied to the same AFE record introduced in the previous sections—coin 12365. For the predicate `nmo:hasIssuer` with the subject `afe:12365`, the presence of the object `ex:uncertain` triggers the uncertainty branch, so the triple « `afe:12365 nmo:hasIssuer ex:uncertain` » gets the weight 0.2, and « `afe:12365 nmo:hasIssuer nm:Augustus` » receives the residual 0.8. And for the predicate `nmo:hasPortrait` with the subject `afe:12365`, exactly one depicted person is asserted and no uncertainty triple exists; therefore, the triple « `afe:12365 nmo:hasPortrait nm:Tiberius` » receives the full weight 1.

For domain-knowledge predicates such as `ex:hasPossibleIssuers`, the absence of an uncertainty triple is always expected. In this case, for the subject `nm:Tiberius` with two objects (`nm:Augustus` and `nm:Tiberius`), the axiom assigns an equal weight of 0.5 to each.

In all cases, as mentioned before the weights for each (s, p) group—i.e., all triples sharing the same subject and predicate—sum to 1.

Additionally, in the *model* column, each triple is assigned the value UR, which is simply the name we gave to the *Uncertainty Reasoner* that produced these results.

s	p	o	weight	model
afe:12365	nmo:hasIssuer	nm:Augustus	0.8	UR
afe:12365	nmo:hasIssuer	ex:uncertain	0.2	UR
afe:12365	nmo:hasPortrait	nm:Tiberius	1	UR
nm:Tiberius	ex:hasPossibleIssuers	nm:Augustus	0.5	UR
nm:Tiberius	ex:hasPossibleIssuers	nm:Tiberius	0.5	UR

Table 4: CertaintyAssignmentAxiom results for AFE record of coin 12365.

In the implementation, *Uncertainty Reasoner* applies this axiom by instantiating it with the chosen predicate p and integrating it into the reasoner’s axiom pipeline. Accordingly, three distinct instances of the axiom are needed for the reasoner to produce the results displayed in Table 4: one for `nmo:hasIssuer`, one for `nmo:hasPortrait`, and another for `ex:hasPossibleIssuers` (see Figure 9).

```
# Query endpoint
QUERY_ENDPOINT = "http://localhost:3030/input/query"
# Update endpoint
UPDATE_ENDPOINT = "http://localhost:3030/input/update"
# Graph Store Protocol endpoint
GSP_ENDPOINT = "http://localhost:3030/input/data"

conn = SparqlConnector.ReificationSparqlConnector(QUERY_ENDPOINT, UPDATE_ENDPOINT, GSP_ENDPOINT)

# define the axiom's pipeline for the Reasoner
axioms = [
    Reasoner.CertaintyAssignmentAxiom("nmo:hasIssuer"),
    Reasoner.CertaintyAssignmentAxiom("nmo:hasPortrait"),
    Reasoner.CertaintyAssignmentAxiom("ex:hasPossibleIssuers"),
]

# Instantiate the Reasoner
reasoner = Reasoner.Reasoner(axioms, reasoner_name="UR")

# load the data
reasoner.load_data_from_endpoint(conn)

# Start the reasoning process
reasoner.reason()
```

Figure 9: Code for running *Uncertainty Reasoner* with `CertaintyAssignmentAxiom` instances in the axiom’s pipeline of the reasoner

3.2.2.2 AFEDempsterShaferAxiom

The Python class `AFEDempsterShaferAxiom` is an axiom that combines the issuers selected by the user with curated domain knowledge—in this case, the possible issuers suggested in the form and defined by experts—which serves as the source of evidence and is processed using *the Dempster’s Rule of Combination* (see Equation 8).

For our use cases, the approach is simplified, because the subsets used typically contain only a single element — the issuer — along with the ignorance set, which represents the set of all possible issuers (Table 5).

To implement this axiom correctly, the triples must first have accurate certainty values. Therefore, `CertaintyAssignmentAxiom` is always applied firstly. Then after that the reasoner applies `AFEDempsterShaferAxiom`, which operates as follows:

1. **User mass function:**

For each coin, the issuer triples are converted into a user mass function. This starts with a default ignorance of 0.2, which is increased by the weight of an explicit “uncertain” issuer triple if present. The remaining mass is then split evenly among the issuers selected by the user.

2. **Domain-knowledge mass functions:**

For each depicted person associated with the coin, a domain-knowledge mass function is constructed. It also starts with a default ignorance of 0.2, which increases by the weight of an explicit “uncertain” depicted-person triple if present, and then distributes the remaining mass evenly across that depicted person’s possible issuers.

3. **Combination and write-back:**

After building these mass functions (one from the user’s selected issuers and one per selected depicted person), they are combined using *the Dempster’s Rule of Combination*. The resulting joint mass values are then written back into the triples: each issuer triple receives its joint mass as the certainty value, and the triple with object `ex:uncertain` and predicate `nmo:hasIssuer` receives the combined ignorance as its new certainty value.

For example, in the same case introduced earlier— coin 12365 —as we can see in Table 5, for the user mass function, because the user selected `Augustus` as the issuer and checked the “uncertain” box, it increased the ignorance value to 0.4 (0.2 default + 0.2 uncertainty weight from Table 4). The remaining certainty of 0.6 was then fully assigned to `Augustus`, as no other issuers were chosen. For the domain-knowledge mass function, only one depicted person was selected, so only one such mass was created. Since the user did not check the “uncertain” box for the selected depicted person, the

ignorance remained at the default 0.2. The remaining 0.8 was split evenly among the possible issuers for that depicted person, resulting in 0.4 for *Augustus* and 0.4 for *Tiberius*.

After computing the joint masses using Dempster's Rule of Combination, the reasoner translates this masses into the final results (see Table 6).

Subset	User Mass	Domain Knowledge Mass Func	Joint Mass
{ <i>Augustus</i> }	0.6	0.4	0.684
{ <i>Tiberius</i> }	0.0	0.4	0.211
Ignorance	0.4	0.2	0.105

Table 5: The mass functions and the joint mass for the AFE record—coin 12365.

s	p	o	weight	model
afe:12365	nmo:hasIssuer	nm:Augustus	0.684	UR
afe:12365	nmo:hasIssuer	nm:Tiberius	0.211	UR
afe:12365	nmo:hasIssuer	ex:uncertain	0.105	UR
afe:12365	nmo:hasPortrait	nm:Tiberius	1	UR
nm:Tiberius	ex:hasPossibleIssuers	nm:Augustus	0.5	UR
nm:Tiberius	ex:hasPossibleIssuers	nm:Tiberius	0.5	UR

Table 6: Final results after applying AFEDempsterShaferAxiom on the AFE record—coin 12365.

It's crucial to understand that the **ignorance** subset, which is utilized in every mass function and has a default value of 0.2, indicates our **level of confidence** in the source that made the choice, in this case the user who selected the issuer and the subject matter experts who established the domain knowledge. Consequently, **higher trust refers to less ignorance**.

Another important point to mention here also is that if a specific coin has neither an issuer nor a depicted person, meaning that the user did not select a value for one or both of these coin's attributes, then the axiom simply ignores it and does not modify its corresponding triples, because the reasoner can not then define a user mass function or a domain knowledge mass function or even both. For this reason, the fifth possibility mentioned earlier in subsection 3.1 is excluded from consideration in this thesis, and only the remaining four possibilities are considered.

Now in the same example—AFE record-coin 12365— if the user had checked the "uncertain" box in the "issuing for" field, the domain-knowledge mass function, as explained earlier, would've also changed from the one in Table 5, where the ignorance would increase to 0.4 and therefore the masses of the issuers *Augustus* and *Tiberius* in the domain-knowledge mass function would decrease (see Table 7).

Subset	User Mass Func	Domain Knowledge Mass Func	Joint Mass
{ <i>Augustus</i> }	0.6	0.3	0.659
{ <i>Tiberius</i> }	0.0	0.3	0.146
Ignorance	0.4	0.4	0.195

Table 7: Mass functions and the joint mass for coin 12365 if the user checked the uncertain box in the "issuing-for" field also.

In the implementation, this axiom is intended to handle multiple predicates within a single instance, in contrast to `CertaintyAssignmentAxiom`, which necessitates a distinct instantiation for every predicate. In addition to the uncertainty marker (`ignorance_object`) and the default uncertainty mass utilized in the mass functions (`ignorance`), the predicates representing the issuer, depicted person, and domain knowledge must be specified in the class parameters (`issuer_predicate`, `issuing_for_predicate`, `domain_knowledge_predicate`).

For the axiom's configuration that led to the results shown in Table 6, these parameters were set to `nmo:hasIssuer`, `nmo:hasPortrait`, `nm:hasPossibleIssuer`, `ex:uncertain`, and 0.2. Similar to `CertaintyAssignmentAxiom`, this axiom needs to be integrated into the reasoner's axiom pipeline after it is instantiated in order to be used during inference (see Figure 10).

```
# Query endpoint
QUERY_ENDPOINT = "http://localhost:3030/input/query"
# Update endpoint
UPDATE_ENDPOINT = "http://localhost:3030/input/update"
# Graph Store Protocol endpoint
GSP_ENDPOINT = "http://localhost:3030/input/data"

conn = SparqlConnector.ReificationSparqlConnector(QUERY_ENDPOINT, UPDATE_ENDPOINT, GSP_ENDPOINT)

# define the axiom's pipeline for the Reasoner
axioms = [
    Reasoner.CertaintyAssignmentAxiom("nmo:hasIssuer"),
    Reasoner.CertaintyAssignmentAxiom("nmo:hasPortrait"),
    Reasoner.CertaintyAssignmentAxiom("ex:hasPossibleIssuers"),
    Reasoner.AFEDempsterShaferAxiom( issuer_predicate: 'nmo:hasIssuer',
                                     issuing_for_predicate: 'nmo:hasPortrait',
                                     domain_knowledge_predicate: 'ex:hasPossibleIssuers',
                                     ignorance_object: 'ex:uncertain')
]

# Instantiate the Reasoner
reasoner = Reasoner.Reasoner(axioms, reasoner_name="UR")

# load the data
reasoner.load_data_from_endpoint(conn)

# Start the reasoning process
reasoner.reason()
```

Figure 10: Code for running *Uncertainty Reasoner* with `CertaintyAssignmentAxiom` and `AFEDempsterShaferAxiom` defined in the axiom's pipeline of the reasoner

4 Classification of AFE Web-Form Input Patterns: issuer & issuing-for

Before introducing the improvements made to *Uncertainty Reasoner*, in this section we inspect how users engage with the AFE web form. Our goal is to first understand the interplay between the fields "issuer" and "issuing-for", then identify the most frequent input scenarios, and recognize significant edge cases that point to areas where this reasoner can be improved. This section presents a simple classification of input patterns, shares their real-world occurrence in AFE, and distinguishes between "positive" and "negative" cases. This analysis motivates the enhancements presented after in section 5.

4.1 Issuer & Depicted Person as One Integrated Use Case: 16 input patterns

It is crucial to understand that the fields *issuer* and *issuing-for* should not be examined separately in real-world situations where users—numismatists—interact with the AFE web form. Given their interdependence, both attributes need to be viewed as a single, integrated use case.

This matches the processing logic of *Uncertainty Reasoner*—specifically the `AFEDempsterShaferAxiom`, which requires input from both fields (the selected issuer(s) and the selected depicted person(s)) together with domain knowledge provided by experts (see subsection 3.2.2.2).

There are four possible input types for each of the two fields, "issuer" and "issuing-for" (the fifth, as previously mentioned, is not taken into account). Therefore, the combination of the four possibilities from each field results in 16 possible input patterns, which are considered in this thesis:

1. Single depicted person + Certain / Single issuer + Certain
2. Single depicted person + Certain / Single issuer + Uncertain
3. Single depicted person + Certain / Multiple issuers + Certain
4. Single depicted person + Certain / Multiple issuers + Uncertain
5. Single depicted person + Uncertain / Single issuer + Certain
6. Single depicted person + Uncertain / Single issuer + Uncertain
7. Single depicted person + Uncertain / Multiple issuers + Certain
8. Single depicted person + Uncertain / Multiple issuers + Uncertain

9. Multiple depicted persons + Certain / Single issuer + Certain
10. Multiple depicted persons + Certain / Single issuer + Uncertain
11. Multiple depicted persons + Certain / Multiple issuers + Certain
12. Multiple depicted persons + Certain / Multiple issuers + Uncertain
13. Multiple depicted persons + Uncertain / Single issuer + Certain
14. Multiple depicted persons + Uncertain / Single issuer + Uncertain
15. Multiple depicted persons + Uncertain / Multiple issuers + Certain
16. Multiple depicted persons + Uncertain / Multiple issuers + Uncertain

After enumerating the 16 issuer $\times$ depicted-person input patterns, we quantified their frequency on real AFE records and two observations stand out:

- the first possibility—single selected issuer marked certain with a single selected depicted person marked also certain—dominates with **3,684** coins, which is about **79.4%** of the analyzed coins in the AFE database.
- the marginal totals reveal asymmetries: **uncertainty** is more frequent on the depicted person side, where **7.5%** of coins have the depicted person marked uncertain (single or multiple), versus **2.2%** on the issuer side. Conversely, **multiplicity** is more common for issuers, where 12.7% of coins have multiple issuers selected (certain or uncertain), versus **3.7%** with multiple depicted persons.

These results are illustrated in the heatmap of Figure 11, where columns correspond to the *issuer* field and rows to *issuing-for* (depicted person). Axis labels use the following abbreviations: **S** = Single (exactly one selected value), **M** = Multiple (more than one alternative selected); **C** = Uncertain checkbox not checked, **U** = Uncertain checkbox checked. Throughout, we refer to cells by their row/column pair (e.g., SC–SC, SC–MC, SU–SC).

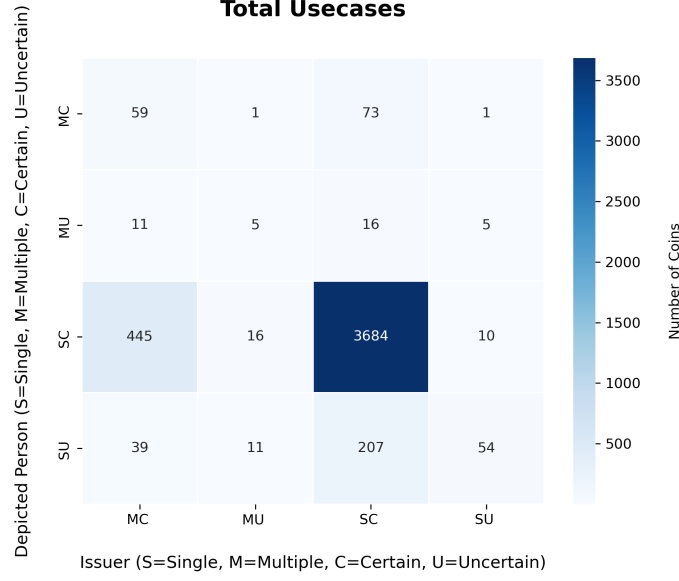


Figure 11: Heatmap of issuer $\times$ depicted-person input patterns in AFE records, with cell values indicating the *number of coins* per pattern.

4.2 Further analysis: Positive and negative use cases

In order to better capture informative edge cases, we divided the 16 possibilities defined earlier into two subtypes, positive and negative.

A case is considered negative when there is a complete mismatch between what the user selected and what the AFE web form suggests from domain knowledge. For instance, the user selects a depicted person, but none of the selected issuer(s) appear in the form’s suggested issuers of that depicted person. But, if there is any overlap, for example, if the user selects at least one issuer that appears in the suggested-issuer list of the selected depicted person, then the case is considered positive.

After dividing all the AFE records from Figure 11 into these two subtypes (see Figure 12), the following finding was noteworthy: **Negative cases are uncommon but concentrated.**

More specifically, 456 AFE coins (9.8%) of the 4,637 examined ones are categorised as negative. The great majority—**430** coins—occur in the **SC–SC** cell (single and certain selected issuer with a single and certain selected depicted person), making up **11.7%** of all **SC–SC** coins and **94.3%** of all negative cases.

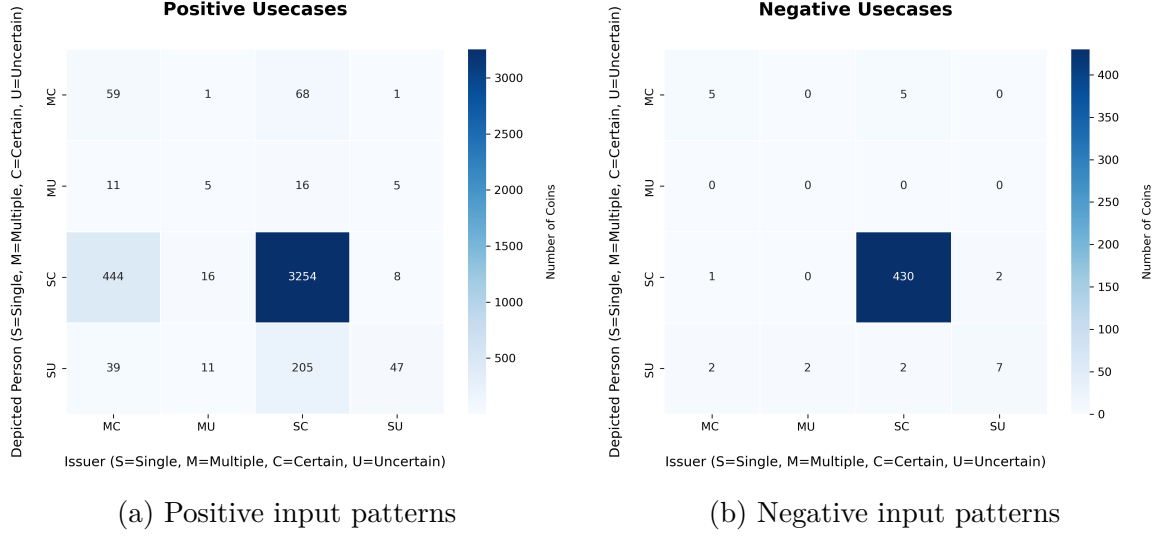


Figure 12: Heatmaps of the issuer $\times$ depicted-person input patterns in AFE records for (a) positive and (b) negative cases.

Out of all the findings discussed in this section, two are particularly important for establishing the improvements of *Uncertainty Reasoner*: the **higher frequency of uncertainty on the depicted-person side** and the concentration of **430 negative cases in the SC–SC cells**. These facts serve as the foundation for the enhancements covered in the following section, which customises the reasoner to produce better outcomes and offer more flexibility.

5 Enhancement of Uncertainty Reasoner

After reviewing the AFE records and discovering interesting patterns in the issuer $\times$ depicted-person inputs, we noticed some interesting results that led us to enhance *Uncertainty Reasoner* and customise it to meet the requirements of the AFE team. In the following three subsections 5.1, 5.2 and 5.3, we cover the enhancements made. Every subsection will outline the issue, the solution we came up with, how it was incorporated into the code, and, if relevant, any limitations.

5.1 First Enhancement: Prioritizing Expert Knowledge in special Negative Cases

5.1.1 Context and Issue

After we further grouped the issuer $\times$ depicted-person input patterns into positive and negative cases and grouped the AFE records into these types of cases, we found out that negative cases are less common than positive cases, but highly concentrated in the SC-SC cell with 430 coins (see subsection 4.2).

These cases are useful for analyzing the behavior of *Uncertainty Reasoner* because they capture situations in which the user's choices contradict experts-based suggestions, yet all of these choices are made with high confidence.

Therefore, we examined a real AFE record—coin 35357—from the 430 coins in the SC-SC cell, by applying *Uncertainty Reasoner* to this case, and analyze its results, if they are plausible.

In Figure 13, the real user's selections made in the fields “issuing for” and “issuer” of the the AFE web form for the coin 35357, are illustrated. In this case, the user identified *Divus Augustus* as the depicted person on the coin. Although the AFE web form suggested multiple possible issuers (*Tiberius*, *Caligula*, *Claudius*, and *Traianus Decius*), the user instead selected *Nerva* as the issuer. Furthermore, as expected from the coins in the SC-SC cell, the uncertain option was left unchecked for both fields, which indicates that the user expressed very high confidence in the correctness of the chosen values.

As explained in subsection 3.2.2.2, before applying the reasoner to the data, we firstly need to define the axioms pipeline that it should follow and use. In our case we defined the four instances of `CertaintyAssignmentAxiom` for each predicate and then one `AFEDempsterShaferAxiom` instance.

Issuing for	Divus Augustus (Divus Augustus) Possible Issuers: - Tiberius (Tiberius) - Caligula (Caligula) - Claudius I (Claudius I.) - Traianus Decius (Traianus Decius)	☐ uncertain
Issuing for alternative		
Issuer	Nerva (Nerva)	☐ uncertain

Figure 13: AFE web-form selections for the coin 35357.

Consequently, using **CertaintyAssignmentAxiom**, *Uncertainty Reasoner* first assigns certainty values to each predicate in the dataset (see Table 8). Following that, it uses the **AFEDempsterShaferAxiom** to apply the Dempster–Shafer Theory, resulting in the outcomes shown in Table 9.

s	p	o	weight	model
afe:35357	nmo:hasIssuer	nm:Nerva	0.8	UR
afe:12365	nmo:hasPortrait	nm:Divus_Augustus	0.8	UR
nm:Divus_Augustus	ex:hasPossibleIssuers	nm:Tiberius	0.25	UR
nm:Divus_Augustus	ex:hasPossibleIssuers	nm:Caligula	0.25	UR
nm:Divus_Augustus	ex:hasPossibleIssuers	nm:Claudius_I	0.25	UR
nm:Divus_Augustus	ex:hasPossibleIssuers	nm:Traianus_Decius	0.25	UR
afe:35357	nmo:hasPortrait	ex:uncertain	0.2	UR
afe:35357	nmo:hasIssuer	ex:uncertain	0.2	UR

Table 8: **CertaintyAssignmentAxiom** results for AFE record— coin 35357

s	p	o	weight	model
afe:35357	nmo:hasIssuer	nm:Caligula	0.111	UR
afe:35357	nmo:hasIssuer	nm:Claudius_I	0.111	UR
afe:35357	nmo:hasIssuer	nm:Traianus_Decius	0.111	UR
afe:35357	nmo:hasIssuer	nm:Tiberius	0.111	UR
afe:35357	nmo:hasIssuer	ex:uncertain	0.111	UR
afe:35357	nmo:hasIssuer	nm:Nerva	0.444	UR
afe:35357	nmo:hasPortrait	ex:uncertain	0.2	UR
afe:35357	nmo:hasPortrait	nm:Divus_Augustus	0.8	UR
nm:Divus_Augustus	ex:hasPossibleIssuers	nm:Tiberius	0.25	UR
nm:Divus_Augustus	ex:hasPossibleIssuers	nm:Caligula	0.25	UR
nm:Divus_Augustus	ex:hasPossibleIssuers	nm:Claudius_I	0.25	UR
nm:Divus_Augustus	ex:hasPossibleIssuers	nm:Traianus_Decius	0.25	UR

Table 9: Final results after applying **AFEDempsterShaferAxiom** on the AFE record—coin 35357

As we can see in the results above, the reasoner assigned to the issuer **Nerva** the highest certainty value of 0.444, while each expert-suggested issuer **Caligula**, **Claudius I**, **Trajanus Decius**, and **Tiberius** received only 0.111 each.

So basically, due to the certainty expressed by the user towards the selected issuer—**Nerva**—even if it is not among the suggested ones established by domain experts, the reasoner ignored that mismatch and assigned the highest certainty value to the selected issuer.

However, **such results are not always ideal** in cases like coin 35357, because the list of suggested issuers is curated by domain experts and is therefore the more reliable source. This raises the question of whether, in such cases, a user’s selection, despite expressing high confidence in both fields, can actually be considered as correct.

The **cause behind this problem** is in the **ignorance mass**, which, as discussed in paragraph 3.2.2.2, measures the degree to which we do not commit to a particular hypothesis and thus represents the level of confidence we have in a source. In the original design of the **AFEDempsterShaferAxiom** (see Subsection 3.2.2.2), both the user and the domain knowledge were given the same ignorance mass of 0.2 through the attribute **ignorance**, which gives both the user and the experts equal reliability (see Table 10).

Subset	User Mass Func	Domain Knowledge Mass Func	Joint Mass
{Nerva}	0.8	0	0.8
{Caligula}	0	0.2	0.111
{Claudius_I}	0	0.2	0.111
{Traianus_Decius}	0	0.2	0.111
{Tiberius}	0	0.2	0.111
ignorance	0.2	0.2	0.111

Table 10: The mass functions and the joint mass for the AFE record—coin 35357—established by the reasoner

Therefore, we had to come up with a solution that, in cases like coin 35357, also enables *Uncertainty Reasoner* to favor the issuers recommended by the experts when AFE team choose so.

5.1.2 Proposed Approach

In order to fix this issue, the user’s ignorance should remain at 0.2, while the domain-knowledge ignorance need to be reduced to a certain level so that was removed from **ignorance** is redistributed equally across the expert-suggested issuers, which increases each issuer’s mass, thereby strengthening the role of expert knowledge in the joint mass calculation.

So in our test, using the same use case— coin 35357 —we reduced the ignorance of the domain-knowledge ignorance from 0.2 to 0.05. And since all mass values must sum to 1, the 0.15 that was removed from the ignorance was redistributed equally across the four expert-suggested issuers, which increased each issuer’s mass from 0.20 to 0.2375. The resulting joint masses (Table 11), translated into the final results (Table 12), shows that the recommendations made by domain experts are given more weight than the contradictory user selected issuer, which made the reasoner to prioritize the issuers recommended by domain experts.

Subset	User Mass Func	Domain Knowledge Mass Func	Joint Mass
{Nerva}	0.8	0	0.167
{Caligula}	0	0.2375	0.198
{Claudius_I}	0	0.2375	0.198
{Traianus_Decius}	0	0.2375	0.198
{Tiberius}	0	0.2375	0.198
ignorance	0.2	0.05	0.042

Table 11: The mass functions and the joint mass for the AFE record—coin 35357—established by the reasoner after reducing the ignorance in the domain knowledge mass function to 0.05.

s	p	o	weight	model
ex:coin_35357	nmo:hasIssuer	ex:uncertain	0.042	UR
ex:coin_35357	nmo:hasIssuer	ex:Nerva	0.167	UR
ex:coin_35357	nmo:hasIssuer	ex:Caligula	0.198	UR
ex:coin_35357	nmo:hasIssuer	ex:Claudius_I	0.198	UR
ex:coin_35357	nmo:hasIssuer	ex:Traianus_Decius	0.198	UR
ex:coin_35357	nmo:hasIssuer	ex:Tiberius	0.198	UR
ex:coin_35357	nmo:hasPortrait	ex:uncertain	0.2	UR
ex:coin_35357	nmo:hasPortrait	ex:Divus_Augustus	0.8	UR
nm:Divus_Augustus	ex:hasPossibleIssuers	nm:Tiberius	0.25	UR
nm:Divus_Augustus	ex:hasPossibleIssuers	nm:Caligula	0.25	UR
nm:Divus_Augustus	ex:hasPossibleIssuers	nm:Claudius_I	0.25	UR
nm:Divus_Augustus	ex:hasPossibleIssuers	nm:Traianus_Decius	0.25	UR

Table 12: Final results after applying AFEDempsterShaferAxiom on the AFE record—coin 35357 and lowering the domain knowledge ignorance to 0.05.

However, because the number expert-suggested issuers (k) varies depending on the depicted person, we cannot rely on a fixed value like 0.05. Rather, given the user’s ignorance (U) and the number of expert suggestions (k) for a selected depicted person, we need to develop a rule for calculating the maximum allowed ignorance mass for the domain knowledge (D), which increases the significance of experts suggestions in the joint mass computation, in cases like coin 35357.

5.1.2.1 Assumptions and setting

First we assume:

- $U \in (0, 1)$ is the user's ignorance mass.
- $x \in \mathbb{N}$, $1 \leq x \leq 3$, is the number of the different issuers selected by the user.
- $D \in [0, 1)$ is the domain-knowledge ignorance mass.
- $k \in \mathbb{N}$, $k \geq 1$, is the number of the different experts-suggested issuers.
- After setting the ignorance mass in each mass function, the **non-ignorance masses are evenly distributed among the corresponding singletons**.
- **None of the expert-suggested issuers and the user-selected issuers align**, as expected in negative cases.

5.1.2.2 Rule for Domain-Knowledge Ignorance in negative cases

So, in order to ensure that expert-suggested issuers receives more weight than each user-selected issuer after applying the Dempster–Shafer rule of combination in the mismatch case, it is sufficient that:

$$D < \frac{Ux}{Ux + k(1 - U)}. \quad (10)$$

5.1.2.3 Proof of the Rule

Using the Dempster–Shafer Combination Rule (Equation 8) and the assumptions mentioned above, the following proof is obtained:

For a user selected issuer u and an expert suggested issuer s_i , let the user's mass function be:

$$m_{\text{user}}(\{u\}) = \frac{1 - U}{x}, \quad m_{\text{user}}(\text{ignorance}) = U$$

and let the experts mass function (domain knowledge mass function) be:

$$m_{\text{expert}}(\{s_i\}) = \frac{1 - D}{k}, \quad m_{\text{expert}}(\text{ignorance}) = D$$

Therefore, for the expert suggested issuer s_i , the corresponding joint mass (before the normalization) is:

$$\tilde{m}(\{s_i\}) = m_{\text{user}}(\text{ignorance}) \cdot m_{\text{expert}}(\{s_i\}) = U \cdot \frac{1 - D}{k}$$

And, for the user selected issuer u , the corresponding joint mass (before the normalization) is:

$$\tilde{m}(\{u\}) = m_{\text{user}}(\{u\}) \cdot m_{\text{expert}}(\text{ignorance}) = \frac{1 - U}{x} \cdot D$$

Because the normalization in the Combination Rule divides by the same factor for both (see Equation 8), the comparison between the two joint masses $m(\{s_i\})$ and $m(\{u\})$ depends only on their numerators $\tilde{m}(\{s_i\})$ and $\tilde{m}(\{u\})$.

Therefore requiring $\tilde{m}(\{s_i\}) > \tilde{m}(\{u_j\})$ yields the following:

$$\begin{aligned} U \cdot \frac{1 - D}{k} &> \frac{1 - U}{x} D \\ \implies \frac{U}{k} - \frac{U}{k} D &> \frac{1 - U}{x} D \\ \implies \frac{U}{k} &> \left(\frac{U}{k} + \frac{1 - U}{x} \right) D \\ \implies D &< \frac{Ux}{Ux + k(1 - U)}. \end{aligned}$$

One note is that $\frac{Ux}{Ux + k(1 - U)}$ is always positive and no sign issues occur in the derivation, because $U \in (0, 1)$ and $k, x \geq 1$, therefore the denominator $Ux + k(1 - U)$ is positive.

5.1.2.4 Evaluation

To evaluate the rule, we tested the same use case described in this section—coin 35357—and the tests are listed as follows:

Test 1:

For the original parameters $U = 0.2$, $x = 1$, and $k = 4$, we obtained:

$$D_{\max} = \frac{0.2}{0.2 + 4 \cdot 0.8} \approx 0.0588.$$

Thus, any $D < 0.0588$ ensures that the experts' opinions dominate. This explains why in our earlier test with $D = 0.05$, the method behaved as expected (see Table 11 and Table 12).

Test 2:

In the second test, we added an additional anonymous possible issuer (for rule verification purposes), so the number of suggested issuers was 5 instead of 4.

Therefore, with $U = 0.2$, $x = 1$, and $k = 5$, we obtained:

$$D_{\max} = \frac{0.2}{0.2 + 5 \cdot 0.8} \approx 0.0476.$$

In this case, `AFEDempsterShaferAxiom` failed to generate the results we wanted when D was set to 0.05. Nevertheless, the anticipated outcomes were attained when D was lowered to 0.04.

Therefore, these tests demonstrate that, in cases like coin 35357, this rule adapts automatically to the number of expert suggestions for a selected depicted person, and the number of selected issuers and find the maximum value allowed for the ignorance mass (D) in the domain knowledge mass function, in order to increase the significance of experts suggestions in the joint mass computation.

5.1.3 Implementation Details

In terms of the implementation, we refactored `AFEDempsterShaferAxiom` class so that the user and expert evidence can carry **different ignorance masses** and, **optionally**, the axiom can **auto-tune** the domain knowledge ignorance in exactly those mismatch cases where the rule applies (see Equation 10 and the related assumptions).

We therefore removed the `ignorance` attribute from `AFEDempsterShaferAxiom` and replaced it with the following:

- `target_ignorance` (default=0.2): represents the ignorance mass in the *user* mass function.
- `domain_knowledge_ignorance` (default=0.2): represents the ignorance mass in the *domain knowledge* mass functions.
- `auto_tune_domain_ignorance` (default=False): if set to `True`, the axiom automatically computes the maximal expert-ignorance $D_{\max}$ allowed by the rule from Equation 10 and uses a rounded-down value of $D_{\max}$ (two decimals) to prioritize experts suggestions for the selected depicted person.

This Auto-tuning, is applied to a use case **only if** it fulfills the assumptions mentioned earlier in subsection 5.1.2.1. Therefore the following conditions must hold:

1. The user's selected issuers and the expert-suggested issuers from the selected depicted person are disjoint, which represents negative cases.

2. The certainty states in both fields ("Issuer" and "Issuing for") are equal and are both either as “certain” or “uncertain” marked.

If these conditions are not satisfied, the axiom falls back to the default ignorance values or to those manually configured by the user, because if the above mentioned conditions are not satisfied then there is no need for the auto-tuning.

This redesign provides a lot of flexibility, depending on our needs, because it allows two different working modes, depending on whether the AFE team prefer to *manually* set the ignorance parameters or let the axiom *automatically* derive them.

- **Option 1 – Prioritizing expert knowledge in special negative cases:**

The AFE team can modify the axiom instance in two ways if they want to treat user-selected issuers as implausible whenever they disagree with expert suggestions:

- *Manual mode:*

According to the introduced rule, set `domain_knowledge_ignorance` to a lower value, like 0.05, while maintaining `target_ignorance` at 0.2 and leaving `auto_tune_domain_ignorance` at its default value (False). With this configuration, the reasoner uses the given ignorance values and prioritizes the issuers suggested by experts, resulting in the same overall behavior as Table 12.

- *Automatic mode:*

Set `auto_tune_domain_ignorance` to True. Then, when a special negative mismatch— like in coin 35357 —that fulfills the auto-tuning conditions occurs, the axiom automatically applies the rule from Equation 10 and calculates the appropriate domain-knowledge ignorance for each selected depicted person without the need for manual tuning; if not, it either uses the ignorance attributes default values or any values the user specify.

- **Option 2 – Preserving the possibility of user correctness in special negative cases:**

if we want to acknowledge that the user choices might still be correct, especially since leaving the uncertain box unchecked in both fields expresses a high confidence, then by leaving both ignorances at their default values of 0.2 and keeping `auto_tune_domain_ignorance` at its default value (False), The axiom work as it normally does. For the use case examined in this thesis— coin 35357 —this configuration produces the same result as in Table 9, where the reasoner originally gives the user-selected issuer precedence over the recommendations of the experts.

In conclusion, the redesigned `AFEDempsterShaferAxiom` operates exactly as before when using ignorance default values or manually setting them. However, it can now

differentiate between user and expert ignorance and additionally, if desired, can automatically apply the rule to prioritize expert suggestions in special negative mismatch scenarios like coin 35357, by just setting the parameter `auto_tune_domain_ignorance` to `True` (see Figure 14).

```
# Query endpoint
QUERY_ENDPOINT = "http://localhost:3030/input/query"
# Update endpoint
UPDATE_ENDPOINT = "http://localhost:3030/input/update"
# Graph Store Protocol endpoint
GSP_ENDPOINT = "http://localhost:3030/input/data"

conn = SparqlConnector.ReificationSparqlConnector(QUERY_ENDPOINT, UPDATE_ENDPOINT, GSP_ENDPOINT)

# define the axiom's pipeline for the Reasoner
axioms = [
    Reasoner.CertaintyAssignmentAxiom("nmo:hasIssuer"),
    Reasoner.CertaintyAssignmentAxiom("nmo:hasPortrait"),
    Reasoner.CertaintyAssignmentAxiom("ex:hasPossibleIssuers"),
    Reasoner.AFEDempsterShaferAxiom_2(issuer_predicate='nmo:hasIssuer',
                                     issuing_for_predicate='nmo:hasPortrait',
                                     domain_knowledge_predicate='ex:hasPossibleIssuers',
                                     ignorance_object='ex:uncertain',
                                     target_ignorance=0.2,
                                     domain_knowledge_ignorance=0.2,
                                     auto_tune_domain_ignorance=True)
]

# Instantiate the Reasoner
reasoner = Reasoner.Reasoner(axioms, reasoner_name="UR")

# load the data
reasoner.load_data_from_endpoint(conn)

# Start the reasoning process
reasoner.reason()
```

Figure 14: Code for running *Uncertainty Reasoner* with `CertaintyAssignmentAxiom` and `AFEDempsterShaferAxiom` defined in the axiom's pipeline of the reasoner and with the auto tuning option set to true for `AFEDempsterShaferAxiom`

5.2 Second Enhancement: Applying AFEDempsterShaferAxiom to the coin's depicted person

5.2.1 Context and Issue

As mentioned in Section 4.1, according to our analysis of the AFE corpus, we found out that the uncertainty is more frequent on the depicted person side (marginal total of 7.5%) than on the issuer side (marginal total of 2.2%). However, AFEDempsterShaferAxiom assigns certainty values exclusively to the issuers. For that reason, addressing uncertainty just for issuers ignores a significant source of noise.

In this thesis, to give concrete form to the issue, we investigated another AFE record—coin 36911—which demonstrates exactly the pattern that challenges the original design of the original AFEDempsterShaferAxiom: high confidence in the selected issuers paired with low confidence in the depicted person.

Issuing for	Divus Claudius II (Divus Claudius II.) Possible Issuers: Claudius II (Claudius II.)	☒ uncertain
Issuing for alternative		
Issuer	Tetricus I (Tetricus I.)	☐ uncertain
Issuer alternative 1	Quintillus (Quintillus)	

Figure 15: AFE web-form selections for the coin 36911.

As shown in Figure 15, the user selected **Divus Claudius II** as the depicted person with low confidence, ignored the single suggestion (**Claudius II**) and instead selected **Tetricus I** and **Quintillus** as issuers with high confidence. The corresponding RDF graph appears in Figure 16 and is converted into the reasoner's internal representation in Table 13.

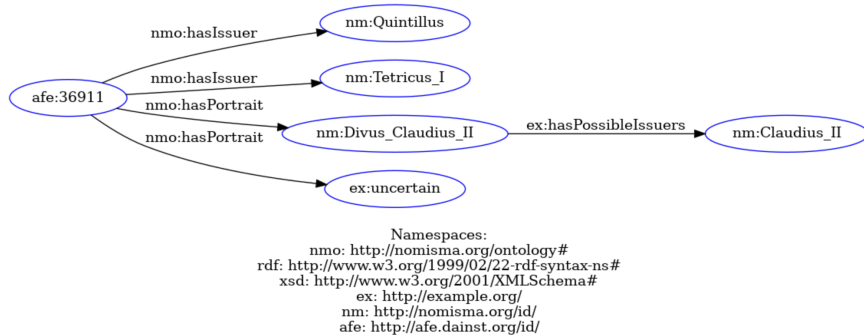


Figure 16: RDF graph representing the entries selected for the AFE record—coin 36911.

s	p	o	weight	model
afe:36911	nmo:hasIssuer	nm:Quintillus	1	
afe:36911	nmo:hasIssuer	nm:Tetricus_I	1	
afe:36911	nmo:hasPortrait	nm:Divus_Claudius_II	1	
afe:36911	nmo:hasPortrait	ex:uncertain	1	
nm:Divus_Claudius_II	ex:hasPossibleIssuers	nm:Claudius_II	1	

Table 13: Internal representation of the RDF graph from Figure 16.

After adding `CertaintyAssignmentAxiom` and `AFEDempsterShaferAxiom` to the reasoner’s pipeline, the `CertaintyAssignmentAxiom` is applied to each triple in the given data, assigning a certainty value to each triple based on its predicate `p` and its subject `s` (see Table 14). Subsequently, the original `AFEDempsterShaferAxiom` applies Dempster’s Rule of Combination to the modified data and infers the final results, as shown in Table 15).

s	p	o	weight	model
afe:36911	nmo:hasIssuer	nm:Quintillus	0.5	UR
afe:36911	nmo:hasIssuer	nm:Tetricus_I	0.5	UR
afe:36911	nmo:hasPortrait	nm:Divus_Claudius_II	0.8	UR
afe:36911	nmo:hasPortrait	ex:uncertain	0.2	UR
nm:Divus_Claudius_II	ex:hasPossibleIssuers	nm:Claudius_II	1	UR

Table 14: `CertaintyAssignmentAxiom` results after applying it to data from Table 13

s	p	o	weight	model
afe:36911	nmo:hasIssuer	ex:uncertain	0.154	UR
afe:36911	nmo:hasIssuer	nm:Claudius_II	0.231	UR
afe:36911	nmo:hasIssuer	nm:Tetricus_I	0.308	UR
afe:36911	nmo:hasIssuer	nm:Quintillus	0.308	UR
afe:36911	nmo:hasPortrait	ex:uncertain	0.2	UR
afe:36911	nmo:hasPortrait	nm:Divus_Claudius_II	0.8	UR
nm:Divus_Claudius_II	ex:hasPossibleIssuers	nm:Claudius_II	1	UR

Table 15: End results after applying `AFEDempsterShaferAxiom` on the data from Table 14- with issuer was the target for this axiom

These results, however, make it evident that the reasoner assigned `Divus Claudius II` a relatively high certainty score of 0.8. We believe that this value should be lower because the user indicated uncertainty about the person portrayed but high confidence in the chosen issuer. Additionally, the selected issuer is also a possible issuer for other depicted persons, suggesting that there are more suitable candidates, or better qualified, for this coin than `Divus Claudius II`.

The **main cause behind this problem** is in the **design** of `AFEDempsterShaferAxiom`. this axiom is asymmetric, meaning it treats only issuers as the target and combines evidence from expert knowledge of possible issuers for the user-selected depicted person; consequently, it cannot perform the inverse task of inferring candidate

portraits from selected issuers. Another issue is that the original data lacks any relation that links an issuer to its possible portraits (see Figure 16), leaving the reasoner with no evidence to examine the user’s depicted person choice.

5.2.2 Proposed Approach

Thereby, adding a valid evidence to the data was the first step in the enhancement process. We did not need to find new domain knowledge; instead, we added an inverse predicate `ex:inPossibleIssuersOf` of the predicate `ex:hasPossibleIssuers` to the existing ontology to enable evidence flow from issuers to depicted persons.

So, for each issuer x that is listed as a possible issuer of person y , we normally only had the triple $\langle \text{nm:person}_y, \text{ex:hasPossibleIssuers}, \text{nm:issuer}_x \rangle$, but now we also added the following triple $\langle \text{nm:issuer}_x, \text{ex:inPossibleIssuersOf}, \text{nm:person}_y \rangle$.

Therefore, in this use case— coin 36911 —as illustrated in Figure 17 and Table 16, after searching further for each of the selected issuers to determine which possible persons they belong to, the original graph was extended. In this extension, **Tetricus I** is a possible issuer only of **Tetricus II**, **Quintillus** is not a possible issuer of any person, and **Claudius II** is only a possible issuer of **Divus Claudius II**.

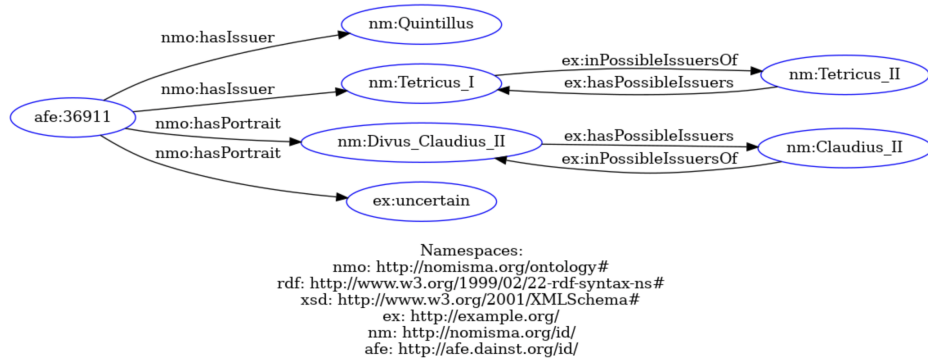


Figure 17: RDF graph representing the entries selected for the AFE record— coin 36911—with additional evidence from the issuers.

s	p	o	weight	model
afe:36911	nmo:hasIssuer	nm:Quintillus	1	
afe:36911	nmo:hasIssuer	nm:Tetricus_I	1	
afe:36911	nmo:hasPortrait	nm:Divus_Claudius_II	1	
afe:36911	nmo:hasPortrait	ex:uncertain	1	
nm:Divus_Claudius_II	ex:hasPossibleIssuers	nm:Claudius_II	1	
nm:Claudius_II	ex:inPossibleIssuersOf	nm:Divus_Claudius_II	1	
nm:Tetricus_I	ex:inPossibleIssuersOf	nm:Tetricus_II	1	
nm:Tetricus_II	ex:hasPossibleIssuers	nm:Tetricus_I	1	

Table 16: Internal representation of the RDF graph from Figure 17.

This data enrichment provides now the necessary evidence to apply Dempster’s rule of combination with depicted persons as the target.

Next, after Applying **CertaintyAssignmentAxiom** on this new modified data (see Table 17), we needed to formalize the logic required to apply the *Combination Rule* to the new target—depicted person.

So, the solution we came up with (see Table 18) is as follows:

- Rather than the issuer, the context is now the depicted person.
- The user mass function represents now the depicted person. It starts with a default ignorance of 0.2. If an explicit depicted person triple with the object **ex:uncertain** is present, its weight is added to the ignorance. The remaining mass is then split evenly among the persons selected by the user.
- The domain knowledge mass function is constructed in a similar way as before, but this time not for each depicted person associated with the coin, but rather for each issuer associated with the coin. It also starts with a default ignorance of 0.2, which increases by the weight of an explicit issuer triple with **ex:uncertain** as the object (if present). The remaining mass is then distributed evenly across that issuer’s possible depicted persons. In our case—coin 35357, the second selected issuer **Quintillus** had no possible depicted persons in the domain knowledge. Therefore, no mass function exists for this issuer.
- After building the required mass functions, they are combined as usual using Dempster’s Rule of Combination. The resulting joint mass values are then written back into the triples (see Table 19): each depicted person triple receives its joint mass as the certainty value, and the triple with the object **ex:uncertain** and the predicate **nmo:hasPortrait** receives the joint mass ignorance value as its new certainty value.

s	p	o	weight	model
afe:36911	nmo:hasIssuer	nm:Quintillus	0.5	UR
afe:36911	nmo:hasIssuer	nm:Tetricus_I	0.5	UR
afe:36911	nmo:hasPortrait	nm:Divus_Claudius_II	0.8	UR
afe:36911	nmo:hasPortrait	ex:uncertain	0.2	UR
nm:Divus_Claudius_II	ex:hasPossibleIssuers	nm:Claudius_II	1	UR
nm:Claudius_II	ex:inPossibleIssuersOf	nm:Divus_Claudius_II	1	UR
nm:Tetricus_I	ex:inPossibleIssuersOf	nm:Tetricus_II	1	UR
nm:Tetricus_II	ex:hasPossibleIssuers	nm:Tetricus_I	1	UR

Table 17: **CertaintyAssignmentAxiom** results after applying it to data from Table 16

context: depicted person	User Mass func	Domain Knowledge Mass func: Tetricus I	Joint Mass
{Divus_Claudius_II}	0.6	0	0.231
{Tetricus_II}	0	0.8	0.615
ignorance	$0.2+0.2 = 0.4$	$0.2+0 = 0.2$	0.154

Table 18: The mass functions and the joint mass for the AFE record— coin 36911—with depicted person as the context (target).

s	p	o	weight	model
afe:36911	nmo:hasPortrait	nm:Divus_Claudius_II	0.231	UR
afe:36911	nmo:hasPortrait	ex:uncertain	0.154	UR
afe:36911	nmo:hasPortrait	nm:Tetricus_II	0.615	UR
afe:36911	nmo:hasIssuer	nm:Quintillus	0.5	UR
afe:36911	nmo:hasIssuer	nm:Tetricus_I	0.5	UR
nm:Divus_Claudius_II	ex:hasPossibleIssuers	nm:Claudius_II	1	UR
nm:Claudius_II	ex:inPossibleIssuersOf	nm:Divus_Claudius_II	1	UR
nm:Tetricus_I	ex:inPossibleIssuersOf	nm:Tetricus_II	1	UR
nm:Tetricus_II	ex:hasPossibleIssuers	nm:Tetricus_I	1	UR

Table 19: Final results after applying `AFEDempsterShaferAxiom` on the AFE record— coin 36911 —with depicted person as the target for this axiom.

Therefore, as these final results demonstrate, using the `AFEDempsterShaferAxiom` also with depicted persons as the target and combining issuer-derived masses via the predicate `ex:inPossibleIssuersOf` with the user mass from `nmo:hasPortrait` results in **Tetricus II** having a certainty value of 0.615, **Divus Claudius II** having a value of 0.231, and an uncertainty of 0.154, which aligns the portrait ranking with the input-confidence pattern—certain selected issuer but uncertain selected depicted person—and **fixes the previous overconfidence** of 0.8 for **Divus Claudius II**.

5.2.3 Implementation Details

In the implementation, several designs were considered. For example, implementing a second axiom class that applies Dempster’s Rule of Combination to the depicted persons, or modifying only the `reason` method of the original `AFEDempsterShaferAxiom` to apply Dempster’s Rule of Combination for both issuers and depicted persons at the same time, using the appropriate evidence for each target. However, these options go against the author’s defined expandability and simplicity properties of *Uncertainty Reasoner* in [13].

Therefore, we refactored the `AFEDempsterShaferAxiom` to make it general, while staying within the AFE context.

First, we replaced the original parameters `issuer_predicate` and `issuing_for_predicate` with two parameters:

- **target_predicate**: the predicate representing the target to which Dempster-Shafer is applied.

Example: we use `nmo:hasIssuer` when the target is the coin’s issuer, and we use `nmo:hasPortrait` when the target is the coin’s depicted person.

- **knowledge_path_predicate**: the predicate that links a coin x to an object o , from which we reach the relevant domain knowledge.

Example: when inferring possible issuers from a selected depicted person, we set `knowledge_path_predicate` to `nmo:hasPortrait`, and when inferring possible depicted persons from a selected issuer, set `knowledge_path_predicate` = `nmo:hasIssuer`.

Then, the other parameters `ignorance_object` and `domain_knowledge_predicate` remained unchanged, as do the previously introduced `target_ignorance`, `domain_knowledge_ignorance` and `auto_tune_domain_ignorance` in subsection 5.1.

Lastly, the `reason` method of `AFEDempsterShaferAxiom` was extended to be parameter-driven, building mass functions from the configured predicates and applying the *Dempster’s Rule of Combination*, instead of being limited to the issuer target.

Thus, with this implementation, the user can define the proper *target*, *evidence-path*, and *domain-knowledge* predicates that best suit their use cases using a single axiom class—`AFEDempsterShaferAxiom` (see Figure 18).

```
# Query endpoint
QUERY_ENDPOINT = "http://localhost:3030/input/query"
# Update endpoint
UPDATE_ENDPOINT = "http://localhost:3030/input/update"
# Graph Store Protocol endpoint
GSP_ENDPOINT = "http://localhost:3030/input/data"

conn = SparqlConnector.ReificationSparqlConnector(QUERY_ENDPOINT, UPDATE_ENDPOINT, GSP_ENDPOINT)

# define the axiom's pipeline for the Reasoner
axioms = [
    Reasoner.CertaintyAssignmentAxiom("nmo:hasIssuer"),
    Reasoner.CertaintyAssignmentAxiom("nmo:hasPortrait"),
    Reasoner.CertaintyAssignmentAxiom("ex:hasPossibleIssuers"),
    Reasoner.CertaintyAssignmentAxiom("ex:inPossibleIssuersOf"),
    Reasoner.AFEDempsterShaferAxiom_2( target_predicate: 'nmo:hasIssuer',
                                     knowledge_path_predicate: 'nmo:hasPortrait',
                                     domain_knowledge_predicate: 'ex:hasPossibleIssuers',)
]

# Instantiate the Reasoner
reasoner = Reasoner.Reasoner(axioms, reasoner_name="UR")

# Load the data
reasoner.load_data_from_endpoint(conn)

# Start the reasoning process
reasoner.reason()
```

(a) Issuer as the target

```
# Query endpoint
QUERY_ENDPOINT = "http://localhost:3030/input/query"
# Update endpoint
UPDATE_ENDPOINT = "http://localhost:3030/input/update"
# Graph Store Protocol endpoint
GSP_ENDPOINT = "http://localhost:3030/input/data"

conn = SparqlConnector.ReificationSparqlConnector(QUERY_ENDPOINT, UPDATE_ENDPOINT, GSP_ENDPOINT)

# define the axiom's pipeline for the Reasoner
axioms = [
    Reasoner.CertaintyAssignmentAxiom("nmo:hasIssuer"),
    Reasoner.CertaintyAssignmentAxiom("nmo:hasPortrait"),
    Reasoner.CertaintyAssignmentAxiom("ex:hasPossibleIssuers"),
    Reasoner.CertaintyAssignmentAxiom("ex:inPossibleIssuersOf"),
    Reasoner.AFEDempsterShaferAxiom_2( target_predicate: 'nmo:hasPortrait',
                                     knowledge_path_predicate: 'nmo:hasIssuer',
                                     domain_knowledge_predicate: 'ex:inPossibleIssuersOf',)
]

# Instantiate the Reasoner
reasoner = Reasoner.Reasoner(axioms, reasoner_name="UR")

# Load the data
reasoner.load_data_from_endpoint(conn)

# Start the reasoning process
reasoner.reason()
```

(b) Depicted person as the target

Figure 18: Code for running *Uncertainty Reasoner* with `CertaintyAssignmentAxiom` and the enhanced `AFEDempsterShaferAxiom` for both targets—issuer (Figure 18a) and depicted person (Figure 18b).

5.3 Third Enhancement: Parallel Execution of the AFEDempsterShaferAxiom

5.3.1 Context and Issue

In the previous section, we introduced the first enhancement, which extended `AFEDempsterShaferAxiom` to apply the **dempster rule of combination** and to reason over both a coin’s issuer and its depicted person as possible targets. During that enhancement, a separate *Uncertainty Reasoner* instance was instantiated for each target, meaning that the axiom was executed independently for the two targets— issuers and depicted persons (see Figure 18).

In practice, however, the AFE team intends to run the axiom on both targets simultaneously through one single *Uncertainty Reasoner* instance, providing a unified set of results. Therefore, during the reasoning process, this one *Uncertainty Reasoner* instance should apply `AFEDempsterShaferAxiom` to both targets and return a coherent and meaningful outcome that combines the issuer and depicted person certainty values generated by this axiom.

So, for example, in the same use case from the previous subsection 5.2— coin 36911— and ideal outcome should be the combination of the issuer certainty results in (Table 15) with the depicted person certainty results in (Table 19), yielding the meaningful results in Table 20.

s	p	o	weight	model
afe:36911	nmo:hasIssuer	ex:uncertain	0.154	UR
afe:36911	nmo:hasIssuer	nm:Claudius_II	0.231	UR
afe:36911	nmo:hasIssuer	nm:Tetricus_I	0.308	UR
afe:36911	nmo:hasIssuer	nm:Quintillus	0.308	UR
afe:36911	nmo:hasPortrait	ex:uncertain	0.154	UR
afe:36911	nmo:hasPortrait	nm:Divus_Claudius_II	0.231	UR
afe:36911	nmo:hasPortrait	nm:Tetricus_II	0.615	UR
nm:Divus_Claudius_II	ex:hasPossibleIssuers	nm:Claudius_II	1	UR
nm:Claudius_II	ex:inPossibleIssuersOf	nm:Divus_Claudius_II	1	UR
nm:Tetricus_I	ex:inPossibleIssuersOf	nm:Tetricus_II	1	UR
nm:Tetricus_II	ex:hasPossibleIssuers	nm:Tetricus_I	1	UR

Table 20: Expected final results, which combine certainty results from both issuer and depicted person targets (derived from Tables 15 and 19).

5.3.1.1 Experiment 1

Therefore, to examine this behavior, we instantiated this time the two `AFEDempsterShaferAxiom` instances with in the same reasoning pipeline (see Figure 19): first one targeting issuers and the other targeting depicted persons.

```
# upload the data from the triple store
conn.upload_df(df_3)

# define the axiom's pipeline for the Reasoner
axioms = [
    Reasoner.CertaintyAssignmentAxiom("nmo:hasIssuer"),
    Reasoner.CertaintyAssignmentAxiom("nmo:hasPortrait"),
    Reasoner.CertaintyAssignmentAxiom("ex:hasPossibleIssuers"),
    Reasoner.CertaintyAssignmentAxiom("ex:inPossibleIssuersOf"),
    Reasoner.AFEDempsterShaferAxiom_2( target_predicate: 'nmo:hasIssuer',
                                       knowledge_path_predicate: 'nmo:hasPortrait',
                                       domain_knowledge_predicate: 'ex:hasPossibleIssuers'),
    Reasoner.AFEDempsterShaferAxiom_2( target_predicate: "nmo:hasPortrait",
                                       knowledge_path_predicate: "nmo:hasIssuer",
                                       domain_knowledge_predicate: "ex:inPossibleIssuersOf")
]

# Instantiate the Reasoner
reasoner = Reasoner.Reasoner(axioms, reasoner_name="UR")
reasoner.load_data_from_endpoint(conn)

# Start the reasoning process
reasoner.reason()
```

Figure 19: Code for running *Uncertainty Reasoner* with `CertaintyAssignmentAxiom` and `AFEDempsterShaferAxiom` for both targets—issuer and depicted person—executed successively within a single pipeline, starting with the issuer followed by the depicted person.

s	p	o	weight	model
afe:36911	nmo:hasPortrait	ex:uncertain	0.113	UR
afe:36911	nmo:hasPortrait	nm:Tetricus_II	0.205	UR
afe:36911	nmo:hasPortrait	nm:Divus_Claudius_II	0.682	UR
afe:36911	nmo:hasIssuer	ex:uncertain	0.154	UR
afe:36911	nmo:hasIssuer	nm:Claudius_II	0.231	UR
afe:36911	nmo:hasIssuer	nm:Tetricus_I	0.308	UR
afe:36911	nmo:hasIssuer	nm:Quintillus	0.308	UR
nm:Divus_Claudius_II	ex:hasPossibleIssuers	nm:Claudius_II	1	UR
nm:Claudius_II	ex:inPossibleIssuersOf	nm:Divus_Claudius_II	1	UR
nm:Tetricus_I	ex:inPossibleIssuersOf	nm:Tetricus_II	1	UR
nm:Tetricus_II	ex:hasPossibleIssuers	nm:Tetricus_I	1	UR

Table 21: Final results produced by the reasoner after applying both axiom instances with in the same pipeline, starting with the issuer followed by the depicted person.

However, *Uncertainty Reasoner* instead produced different values (Table 21) from the expected ones (Table 20) at the level of depicted persons.

The observed **discrepancy** in these results stem from *Uncertainty Reasoner*'s design. According to it, after selecting which axioms to apply and instantiating them, the reasoner writes intermediate results back into the dataset after each axiom execution, which can affect subsequent reasoning steps and modifies the final results.

The **CertaintyAssignmentAxiom** is not influenced by this, since each instance operates on a single predicate and assigns certainties independently (see Section 3.2.2.1).

The two **AFEDempsterShaferAxiom** instances in our test, on the other hand, are not independent given that both read and update triples with the predicates **nmo:hasIssuer** and **nmo:hasPortrait**. Consequently, the second run operates on a modified data produced by the first **AFEDempsterShaferAxiom** instance, so its masses and combined certainty values differ from the expected ones.

Therefore, we looked further at how the two targets interact when the reasoner successively executes both **AFEDempsterShaferAxiom** instances in order to discover why the *combined run* results (Table 21) deviates from the *expected union* results (Table 20).

After the reasoner applies the four **CertaintyAssignmentAxiom** instances, we obtain the same intermediate data as in Table 17. the reasoner then runs the first **AFEDempsterShaferAxiom** instance with issuer as the target and writes its results back, reproducing the same data as in Table 15. At this stage, the graph has already been modified to include an inferred issuer, **Claudius II**, with the weight 0.231, and an issuer-side uncertainty triple, with the weight 0.154, in addition to the two user-selected issuers, **Tetricus I** and **Quintillus** (see Table 22)

s	p	o	weight	model
afe:36911	nmo:hasIssuer	nm:Tetricus_I	0.308	UR
afe:36911	nmo:hasIssuer	nm:Quintillus	0.308	UR
afe:36911	nmo:hasIssuer	nm:Claudius_II	0.231	UR
afe:36911	nmo:hasIssuer	ex:uncertain	0.154	UR
afe:36911	nmo:hasPortrait	nm:Divus_Claudius_II	0.8	UR
afe:36911	nmo:hasPortrait	ex:uncertain	0.2	UR
nm:Divus_Claudius_II	ex:hasPossibleIssuers	nm:Claudius_II	1	UR
nm:Claudius_II	ex:inPossibleIssuersOf	nm:Divus_Claudius_II	1	UR
nm:Tetricus_I	ex:inPossibleIssuersOf	nm:Tetricus_II	1	UR
nm:Tetricus_II	ex:hasPossibleIssuers	nm:Tetricus_I	1	UR

Table 22: Intermediate results after applying the **AFEDempsterShaferAxiom** first instance with issuers as the target

These intermediate results after the reasoner applied the first instance of **AFEDempsterShaferAxiom**, will be used then as the input data for the second **AFEDempsterShaferAxiom** instance.

Therefore, in terms of functionality, **the second instance acts** as if the user checked

the *uncertain* box and chose three issuers, **neither of which is true**, which lead to some changes in the mass functions built in the second instance:

- In addition to the user mass function over the depicted persons, the axiom now builds two domain-knowledge mass functions: one from **Tetricus I** and a new one from **Claudius II** (see Table 23 and compare it with Table 18).
- The issuer-side uncertainty (0.154) is added to the default ignorance (0.2), so each domain mass carries now 0.354 ignorance instead of 0.2 (see Table 23 and compare it with Table 18).

context: depicted person	User Mass func	Domain Knowledge Mass func: Tetricus I	Domain Knowledge Mass func: Claudius II	Joint Mass
{Divus_Claudius_II}	0.6	0	0.646	0.682
{Tetricus_II}	0	0.646	0	0.205
ignorance	$0.2+0.2 = 0.4$	$0.2+0.154 = 0.354$	$0.2+0.154 = 0.354$	0.113

Table 23: The mass functions and the joint mass with depicted person as the target depending on the intermediate results from Table 22.

Given that two domain masses now exists, that support **Divus Claudius II** but only one that supports **Tetricus II**, the joint masses yield values of 0.682 for **Divus Claudius II**, 0.205 for **Tetricus II**, and 0.113 for the ignorance (Table 23). These values are then included in the final results showed earlier in Table 21, leading to differences compared to what we expect in the final results.

This causes a **contradiction**, because while the reasoner **still favors** the issuers **Tetricus I** and **Quintillus**, on the depicted person side it **now favors** **Divus Claudius II**, who **is not supported by those issuers in the domain knowledge** base. Consequently, the reasoner fails to leverage the user’s high-confidence in the issuer selections and therefore failed to assign **Tetricus II** the higher certainty value (see final results in Table 21).

5.3.1.2 Experiment 2

Because we observed that the results are also influenced by the order of the **AFEDempsterShaferAxiom** instances, we repeated the experiment with same usecase—coin 36911—but this time we swapped the order of both instances (Figure 20).

In this experiment, after the reasoner applies the four **CertaintyAssignmentAxiom** instances, the data match Table 17 like in the first experiment. Next, the **AFEDempsterShaferAxiom** instance with the depicted persons as the target was executed first and reproduced Table 19 as expected, since it was applied to the modified data produced by the **CertaintyAssignmentAxiom** instances, which we usually expect.


```

# define the axiom's pipeline for the Reasoner
axioms = [
    Reasoner.CertaintyAssignmentAxiom("nmo:hasIssuer"),
    Reasoner.CertaintyAssignmentAxiom("nmo:hasPortrait"),
    Reasoner.CertaintyAssignmentAxiom("ex:hasPossibleIssuers"),
    Reasoner.CertaintyAssignmentAxiom("ex:inPossibleIssuersOf"),
    Reasoner.AFEDempsterShaferAxiom_2( target_predicate: "nmo:hasPortrait",
                                     knowledge_path_predicate: "nmo:hasIssuer",
                                     domain_knowledge_predicate: "ex:inPossibleIssuersOf"),
    Reasoner.AFEDempsterShaferAxiom_2( target_predicate: 'nmo:hasIssuer',
                                     knowledge_path_predicate: 'nmo:hasPortrait',
                                     domain_knowledge_predicate: 'ex:hasPossibleIssuers')
]

# Instantiate the Reasoner
reasoner = Reasoner.Reasoner(axioms, reasoner_name="UR")
reasoner.load_data_from_endpoint(conn)

# Start the reasoning process
reasoner.reason()

```

Figure 20: Code for running *Uncertainty Reasoner* with *CertaintyAssignmentAxiom* and *AFEDempsterShaferAxiom* for both targets—issuer and depicted person—executed successively within a single pipeline, starting with the depicted person followed by the issuer.

After that, the reasoner executes the second *AFEDempsterShaferAxiom* instance with the issuer as the target. However, this instance is now applied to the previously modified data (Table 19), where the coin is after the modification linked to two depicted person triples—*Divus Claudius II* with a certainty value of 0.231 and *Tetricus II* with 0.615—together with an uncertainty triple of 0.154.

Functionally, this means that the second *AFEDempsterShaferAxiom* instance **will behave** as if the user had originally **provided two portraits** along with an **uncertainty flag, which is not actually the case**. Therefore, the reasoner again produced in the final results (Table 24) **different values** from the expected ones (Table 20), but this time at the issuer level.

Again, to trace the effect of the updated input, the Table 25, which illustrates the masses built by the second instance, details how the intermediate results (Table 19) that servers as the input of this second instance translates into evidence weights:

- the reasoner now creates two domain-knowledge masses in addition to the user mass over issuers: one from *Divus Claudius II* and a new one from *Tetricus II*.
- For each domain-knowledge mass function, the total ignorance is 0.354, which is the sum of the default ignorance (0.2) and the depicted-person uncertainty (0.154). When these are combined into the joint mass, *Tetricus I* receives support from both the user mass function and the *Tetricus II* domain mass

function, yielding the highest certainty value (0.608). **Quintillus** follows with 0.163, supported only by the user mass function, while **Claudius II** ranks third with 0.148.

s	p	o	weight	model
afe:36911	nmo:hasIssuer	ex:uncertain	0.081	UR
afe:36911	nmo:hasIssuer	nm:Claudius_II	0.148	UR
afe:36911	nmo:hasIssuer	nm:Quintillus	0.163	UR
afe:36911	nmo:hasIssuer	nm:Tetricus_I	0.608	UR
afe:36911	nmo:hasPortrait	ex:uncertain	0.154	UR
afe:36911	nmo:hasPortrait	nm:Divus_Claudius_II	0.231	UR
afe:36911	nmo:hasPortrait	nm:Tetricus_II	0.615	UR
nm:Divus_Claudius_II	ex:hasPossibleIssuers	nm:Claudius_II	1	UR
nm:Claudius_II	ex:inPossibleIssuersOf	nm:Divus_Claudius_II	1	UR
nm:Tetricus_I	ex:inPossibleIssuersOf	nm:Tetricus_II	1	UR
nm:Tetricus_II	ex:hasPossibleIssuers	nm:Tetricus_I	1	UR

Table 24: Final results produced by the reasoner after applying both axiom instances with in the same pipeline, starting with the depicted person followed by the issuer.

context: issuers	User Mass func	Domain Knowledge Mass func: Divus Claudius II	Domain Knowledge Mass func : Tetricus II	Joint Mass
Quintillus	0.4	0	0	0.163
Tetricus_I	0.4	0	0.646	0.608
Claudius_II	0	0.646	0	0.148
ignorance	$0.2+0 = 0.2$	$0.2+0.154 = 0.354$	$0.2+0.154 = 0.354$	0.081

Table 25: The mass functions and the joint mass with depicted person as the target depending on the intermediate results from Table 19.

This explains why, in the final results of this experiment, **Tetricus I** and **Quintillus** are no longer equal (unlike the expectation in Table 20). The first pass added **Tetricus II** as a selected depicted person into the data, which in turn provided extra support to the issuer **Tetricus I** but not to **Quintillus**. The imbalance here is, again, an order effect caused by successive execution with in-place updates of the data.

5.3.1.3 Problem Summary

We conclude from the two experiments that when multiple **AFEDempsterShaferAxiom** instances, running on the same predicates but with different targets, are placed in the same pipeline and run sequentially by the reasoner, the expected results are not obtained. The disparities always occur at the level of the second target predicate of **AFEDempsterShaferAxiom**: the problem occurred with depicted persons in the first Experiment and with issuers in the second Experiment.

This happens because the second **AFEDempsterShaferAxiom** instance work on a data

that was already altered by the first `AFEDempsterShaferAxiom` instance, rather than the data generated by the `CertaintyAssignmentAxiom` instance. Consequently, certainty values are changed and new triples are added, therefore the evidence used is manipulated, which may eventually produce unexpected outcomes.

5.3.2 Proposed Approach

As a solution to prevent this kind of issues, we suggest applying all related `AFEDempsterShaferAxiom` instances **in parallel** instead of sequentially. The reasoning process would then proceed as follows:

1. All instances of `CertaintyAssignmentAxiom` are applied first by the reasoner. This generates the independent and clean evidence base that should be used as *Dempster–Shafer Rule of Combination*’s input.
2. Issuer-targeted and depicted-person-targeted `AFEDempsterShaferAxiom` instances are run concurrently on the same unmodified input, rather than sequentially. As a result, each instance builds its mass functions independently of the other’s intermediate results, relying on user input and homogeneous domain knowledge.
3. Every instance of `AFEDempsterShaferAxiom` generates its own output, affecting only the triples associated with its target predicate (`nmo:hasIssuer` or `nmo:hasPortrait`).
4. Finally, the reasoner combines the results by gathering only the **affected triples**—that is, the new triples brought about by the axiom or the existing triples whose certainty values were modified—from each instance. The outcomes can be consistently merged into a single, cohesive dataset since each instance operates on a disjoint target predicate.

For instance, when applying this approach to the same case—coin 36911—the parallel execution generates the issuer-level results from Table 15 and the depicted-person-level results from Table 19. After that, the merge step integrates these into the unified dataset shown in Table 20, which guarantees consistent, trustworthy results and is in line with the desired outcome.

Therefore, this approach ensures that:

- The same evidence basis (the `CertaintyAssignmentAxiom` output) is used to process both targets.
- The outcomes are unaffected by the order of `AFEDempsterShaferAxiom` instances.

5.3.3 Implementation Details

In the implementation, to preserve the reasoner's simplicity and extensibility, we introduced a lightweight grouping mechanism, which consists of some enhancements in *Uncertainty Reasoner* itself and the `AFEDempsterShaferAxiom` also.

The modifications are as follows:

- **Grouping attribute:**

The `AFEDempsterShaferAxiom` now has a `group` attribute (default: `"undefined"`). All instances that should be executed in parallel are given the same group name (e.g., `issuer_issuingFor`). And if no group was assigned to the instance (`group = "undefined"`), it is treated as ungrouped and is carried out in the standard successive procedure.

- **Reasoner execution:**

the reasoner still walks the pipeline in order, but now when it reaches the first `AFEDempsterShaferAxiom` instance that belongs to a specific group, it collects all subsequent instances with the same group and evaluates them in parallel. The reasoning pipeline proceeds once all grouped instances have completed and their results have been merged and written back.

After the implementation of this parallel execution modification, we tested it using the same data and pipeline as in the case study (Figure 21). Both `AFEDempsterShaferAxiom` instances (with issuer target and portrait target) were assigned the same group (`group = "issuingForAndIssuer"`).

In this test, the reasoner executed the `CertaintyAssignmentAxiom` instances sequentially as usual, then applied both `AFEDempsterShaferAxiom` instances in parallel. Thus, the input was the same for every instance (Table 17), the issuer results matched Table 15, the portrait results matched Table 19, and the final results matched the expected union of both sub-results (Table 20).

In other tests, we ungrouped both instances (both had the default value `group = "undefined"`), the reasoner executed them in order, writing back after the first pass. The second pass therefore consumed the modified data, reproducing the order-dependent outcomes (Tables 22 and 25) that caused the change.

These tests demonstrate that grouping both `AFEDempsterShaferAxiom` instances eliminates order effects and produces the desired combined result by forcing both instances to operate over the same immutable input.

```

# define the axiom's pipeline for the Reasoner
axioms = [
    Reasoner.CertaintyAssignmentAxiom("nmo:hasIssuer"),
    Reasoner.CertaintyAssignmentAxiom("nmo:hasPortrait"),
    Reasoner.CertaintyAssignmentAxiom("ex:hasPossibleIssuers"),
    Reasoner.CertaintyAssignmentAxiom("ex:inPossibleIssuersOf"),
    Reasoner.AFEDempsterShaferAxiom_2( target_predicate: "nmo:hasPortrait",
                                       knowledge_path_predicate: "nmo:hasIssuer",
                                       domain_knowledge_predicate: "ex:inPossibleIssuersOf",
                                       group="issuingForAndIssuer"),
    Reasoner.AFEDempsterShaferAxiom_2( target_predicate: 'nmo:hasIssuer',
                                       knowledge_path_predicate: 'nmo:hasPortrait',
                                       domain_knowledge_predicate: 'ex:hasPossibleIssuers',
                                       group="issuingForAndIssuer")
]

# Instantiate the Reasoner
reasoner = Reasoner.Reasoner(axioms, reasoner_name="UR")
reasoner.load_data_from_endpoint(conn)

# Start the reasoning process
reasoner.reason()

df_dempster_result = reasoner.get_triples_as_df()

```

Figure 21: Code for running *Uncertainty Reasoner* with `CertaintyAssignmentAxiom` and parallel `AFEDempsterShaferAxiom` instances for both targets—issuer and depicted person—within a single pipeline.

In addition, this design enhances the reasoner’s performance, which is another important consideration. the reasoner avoids an extra read–write cycle and prevents the usage of redundant triples by combining the two `AFEDempsterShaferAxiom` instances (issuer and depicted person) and evaluating them simultaneously.

In another test (Figure 22), designed to validate the performance improvement, the reasoner that applied the `AFEDempsterShaferAxiom` instances in parallel, not only produced the expected, order-independent results, but even finished the reasoning process across all different dataset sizes **faster** than the the reasoner that applied both instances successively.

Therefore, even if this design gives users precise control by letting them choose which `AFEDempsterShaferAxiom` instances belong to the same group and should be evaluated simultaneously, for AFE use cases like those covered in this thesis, we recommend placing the two `AFEDempsterShaferAxiom` instances in the same group, because it don’t just enhances the runtime of *Uncertainty Reasoner* but it also eliminates inconsistencies and unexpected results.

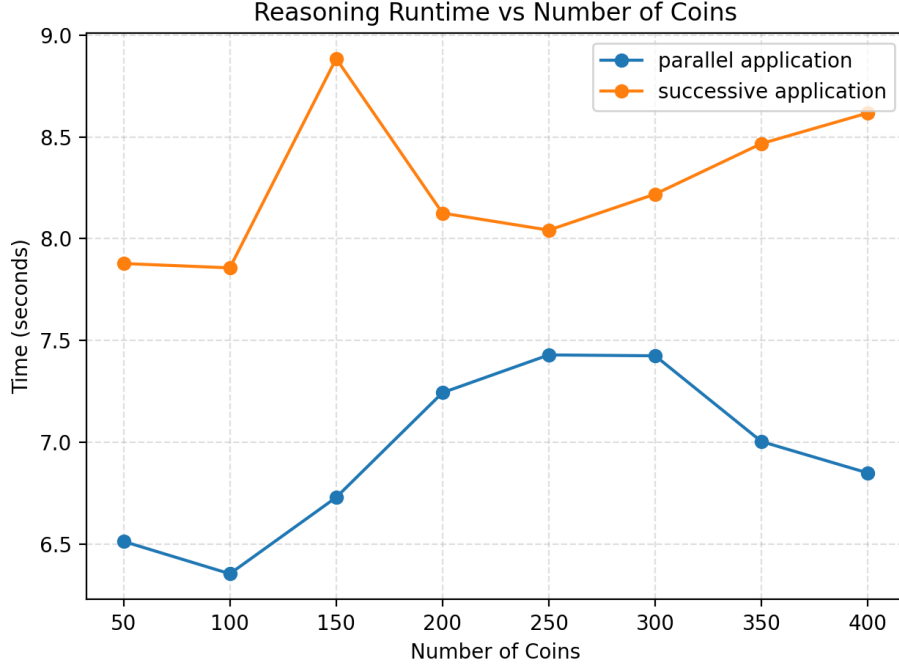


Figure 22: Comparison of reasoning runtime of *Uncertainty Reasoner* for parallel and successive applications of `AFEDempsterShaferAxiom` instances across different size of datasets defined by the number of coins.

5.3.4 Limitations

In this enhancement, there is still one restriction, though. Although the reasoner can apply all `AFEDempsterShaferAxiom` instances of the same group in parallel, we only used two instances of the `AFEDempsterShaferAxiom` in this thesis because we were concentrating on two aspects of the AFE web form—the coin’s issuer and its portrait.

Although the tests performed as anticipated in this environment, it is unknown whether they will still be accurate in configurations with more than two instances because we didn’t had any use cases where we needed them and therefore didn’t assess them.

Future work should therefore extend *Uncertainty Reasoner* to correctly handle more than two `AFEDempsterShaferAxiom` instances in parallel, since AFE team may require additional instances for further AFE requirements.

6 Conclusion

This thesis focused on enhancing the behavior of *Uncertainty Reasoner* towards uncertain and conflicting data by applying it to real records from the Antike Fundmünzen in Europa (AFE) database, represented as RDF graphs.

Through an analysis of the AFE input patterns and their classification into positive and negative cases, several significant discoveries and improvements were achieved.

First, after discussions with the AFE team and domain experts, the analysis showed that there were false and inconsistent entries in some of the 430 negative records in the SC-SC cell, where most of negative cases are concentrated. This demonstrates how the thesis helped to find and fix data errors within AFE itself. But now that the first improvement has been made, the reasoner is redesigned so that, in situations like those in the SC-SC cell, the certainty value is no longer mistakenly biased towards the user-selected issuer. Instead, even if the user expressed his high confidence in the selected values, expert recommendations can be given priority and the results will more accurately reflect the domain knowledge, if the AFE team choose so. The same discussions also showed that the AFE domain knowledge contains some inconsistencies. But updating the domain knowledge is a more complex task that is outside the bounds of this work.

Second, the thesis expanded the reasoner's scope by enabling it to also apply AFEDempsterShaferAxiom to the coin's depicted person, which was previously ignored. Therefore, this is a significant step in the direction of a more improved handling and modeling of uncertainty in numismatic records. Furthermore, AFEDempsterShaferAxiom was redesigned and parallelized, which enhanced reasoning performance in addition to removing order effects.

However, there is still a drawback: only two AFEDempsterShaferAxiom instances were used to test the new parallelization feature. Future AFE requirements might demand configurations with more than two instances, but it is unclear if the current solution will scale to those configurations reliably. Therefore, future work should prioritize addressing this limitation.

Lastly, this thesis has demonstrated how difficult it is to apply the Dempster-Shafer rule of combination in the context of AFE. The behavior of the mass functions can be directly influenced by a wide range of factors, including the number of selected issuers or depicted persons, the use of the uncertainty checkbox, and discrepancies between the suggested and selected values. However, this thesis showed how *Uncertainty Reasoner* can be systematically examined, enhanced, and verified in practice by focusing on specific AFE requirements and crucial use cases.

References

- [1] *AFE Web*. <http://www.bigdata.uni-frankfurt.de/afe-web/>. last accessed 25.09.2025.
- [2] Hamdi Aloulou et al. “Uncertainty handling in semantic reasoning for accurate context understanding”. In: *Knowledge-Based Systems* 77 (2015), pp. 16–28.
- [3] *Antike Fundmünzen in Europa – Suchportal*. URL: <http://afe.dainst.org/search-container/suche>.
- [4] *Corpus Nummorum Portal*. <https://www.corpus-nummorum.eu/>. last accessed 01.09.2025.
- [5] “CORPUS NUMMORUM – A Digital Research Infrastructure for Ancient Coins”. In: (2024). URL: <https://zenodo.org/records/10633905>.
- [6] Pascal Hitzler, Markus Krotzsch, and Sebastian Rudolph. *Foundations of semantic web technologies*. Chapman and Hall/CRC, 2009.
- [7] Pascal Hitzler et al. *Semantic Web: Grundlagen*. Springer, 2007.
- [8] Jan Luca Pöpperl. “Modellierung von Unsicherheiten in Daten: Benchmarktests verschiedener Ansätze”. Master Thesis. MA thesis. Goethe University Frankfurt am Main, 2023. URL: https://github.com/UncertaintyC/unco/blob/42a561ca68543ffbde32e953cd71f18093628824/master_thesis.pdf.
- [9] *RDF 1.1 Semantics – Reification*. URL: <https://www.w3.org/TR/rdf11-mt/#reification>.
- [10] *RDF-star and SPARQL-star*. URL: <https://www.w3.org/2021/12/rdf-star.html>.
- [11] Kari Sentz and Scott Ferson. “Combination of evidence in Dempster-Shafer theory”. In: (2002).
- [12] Karsten Tolle and David Wigg-Wolf. “Uncertainty handling for ancient coinage”. In: (2015), pp. 171–178.
- [13] Jens Weigel. “Uncertainty Reasoner for RDF Graphs based on Numismatic Use Cases”. Master Thesis. MA thesis. Goethe University Frankfurt am Main, 2024. URL: <https://github.com/jensw99/uncertainty-reasoner/blob/main/master-thesis.pdf>.

List of Tables

1	Internal schema for representing RDF graphs with uncertainty (adapted from [13]).	15
2	Internal representation of class relationships in RDF graphs (adapted from [13]).	15
3	Internal Representaion of the RDF graph from Figure 8 for the coin 12365.	16
4	<code>CertaintyAssignmentAxiom</code> results for AFE record of coin 12365. . . .	18
5	The mass functions and the joint mass for the AFE record—coin 12365.	20
6	Final results after applying <code>AFEDempsterShaferAxiom</code> on the AFE record—coin 12365.	20
7	Mass functions and the joint mass for coin 12365 if the user checked the uncertain box in the "issuing-for" field also.	21
8	<code>CertaintyAssignmentAxiom</code> results for AFE record— coin 35357 . . .	27
9	Final results after applying <code>AFEDempsterShaferAxiom</code> on the AFE record—coin 35357	27
10	The mass functions and the joint mass for the AFE record—coin 35357—established by the reasoner	28
11	The mass functions and the joint mass for the AFE record—coin 35357—established by the reasoner after reducing the ignorance in the domain knowledge mass function to 0.05.	29
12	Final results after applying <code>AFEDempsterShaferAxiom</code> on the AFE record—coin 35357 and lowering the domain knowledge ignorance to 0.05.	29
13	Internal representation of the RDF graph from Figure 16.	36
14	<code>CertaintyAssignmentAxiom</code> results after applying it to data from Table 13	36
15	End results after applying <code>AFEDempsterShaferAxiom</code> on the data from Table 14- with issuer was the target for this axiom	36
16	Internal representation of the RDF graph from Figure 17.	37
17	<code>CertaintyAssignmentAxiom</code> results after applying it to data from Table 16	38

18	The mass functions and the joint mass for the AFE record— coin 36911— with depicted person as the context (target).	39
19	Final results after applying <code>AFEDempsterShaferAxiom</code> on the AFE record— coin 36911 —with depicted person as the target for this ax- iom.	39
20	Expected final results, which combine certainty results from both issuer and depicted person targets (derived from Tables 15 and 19).	41
21	Final results produced by the reasoner after applying both axiom in- stances with in the same pipeline, starting with the issuer followed by the depicted person.	42
22	Intermediate results after applying the <code>AFEDempsterShaferAxiom</code> first instance with issuers as the target	43
23	The mass functions and the joint mass with depicted person as the target depending on the intermediate results from Table 22.	44
24	Final results produced by the reasoner after applying both axiom in- stances with in the same pipeline, starting with the depicted person followed by the issuer.	46
25	The mass functions and the joint mass with depicted person as the target depending on the intermediate results from Table 19.	46
26	RDF namespaces that are used in this thesis and other ones.	57

List of Figures

1	Example RDF graph with URI nodes in the AFE domain.	3
2	Same RDF graph as in Figure 1, expressed with prefixes instead of full URIs	5
3	Turtle serialization of the RDF graph from Figure 2	5
4	RDF/XML serialization of the RDF graph from Figure 2	6
5	Uncertainty modeling in RDF	7
6	Uncertainty modeling in RDF	12
7	Uncertainty modeling in RDF	12
8	Uncertainty modeling in RDF	16
9	Code for running <i>Uncertainty Reasoner</i> with <i>CertaintyAssignmentAxiom</i> instances in the axiom’s pipeline of the reasoner	18
10	Code for running <i>Uncertainty Reasoner</i> with <i>CertaintyAssignmentAxiom</i> and <i>AFEDempsterShaferAxiom</i> defined in the axiom’s pipeline of the reasoner	21
11	Heatmap of issuer $\times$ depicted-person input patterns in AFE records, with cell values indicating the <i>number of coins</i> per pattern.	24
12	Heatmaps of the issuer $\times$ depicted-person input patterns in AFE records for (a) positive and (b) negative cases.	25
13	AFE web-form selections for the coin 35357.	27
14	Code for running <i>Uncertainty Reasoner</i> with <i>CertaintyAssignmentAxiom</i> and <i>AFEDempsterShaferAxiom</i> defined in the axiom’s pipeline of the reasoner and with the auto tuning option set to true for <i>AFEDempsterShaferAxiom</i>	34
15	AFE web-form selections for the coin 36911.	35
16	RDF graph representing the entries selected for the AFE record—coin 36911.	35
17	RDF graph representing the entries selected for the AFE record—coin 36911—with additional evidence from the issuers.	37

18	Code for running <i>Uncertainty Reasoner</i> with <code>CertaintyAssignmentAxiom</code> and the enhanced <code>AFEDempsterShaferAxiom</code> for both targets—issuer (Figure 18a) and depicted person (Figure 18b).	40
19	Code for running <i>Uncertainty Reasoner</i> with <code>CertaintyAssignmentAxiom</code> and <code>AFEDempsterShaferAxiom</code> for both targets—issuer and depicted person—executed successively within a single pipeline, starting with the issuer followed by the depicted person.	42
20	Code for running <i>Uncertainty Reasoner</i> with <code>CertaintyAssignmentAxiom</code> and <code>AFEDempsterShaferAxiom</code> for both targets—issuer and depicted person—executed successively within a single pipeline, starting with the depicted person followed by the issuer.	45
21	Code for running <i>Uncertainty Reasoner</i> with <code>CertaintyAssignmentAxiom</code> and parallel <code>AFEDempsterShaferAxiom</code> instances for both targets—issuer and depicted person—within a single pipeline.	49
22	Comparison of reasoning runtime of <i>Uncertainty Reasoner</i> for parallel and successive applications of <code>AFEDempsterShaferAxiom</code> instances across different size of datasets defined by the number of coins.	50

Appendices

A. RDF Namespaces

Prefix	Namespace
rdf	http://www.w3.org/1999/02/22-rdf-syntax-ns#
rdfs	http://www.w3.org/2000/01/rdf-schema#
owl	http://www.w3.org/2002/07/owl#
nm	http://nomisma.org/id/
nmo	http://nomisma.org/ontology#
ex	http://example.com/

Table 26: RDF namespaces that are used in this thesis and other ones.